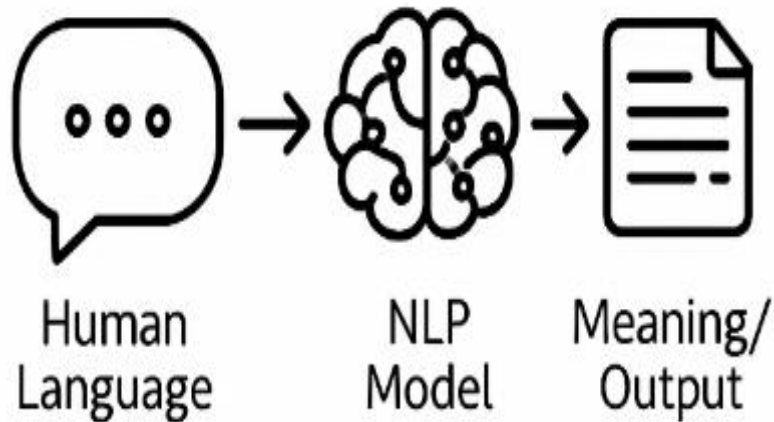


NATURAL LANGUAGE PROCESSING (NLP)

PMDS606L



MODULE 4

Dr. Reya Sharma

Assistant Professor

Dept. of Analytics, SCOPE, VIT

Parts-of-Speech Tagging

Part-of-Speech tagging is the process of assigning a grammatical category (like nouns, verbs, adjectives, adverbs, pronouns, prepositions, etc.) to each word in a sentence.

For example:

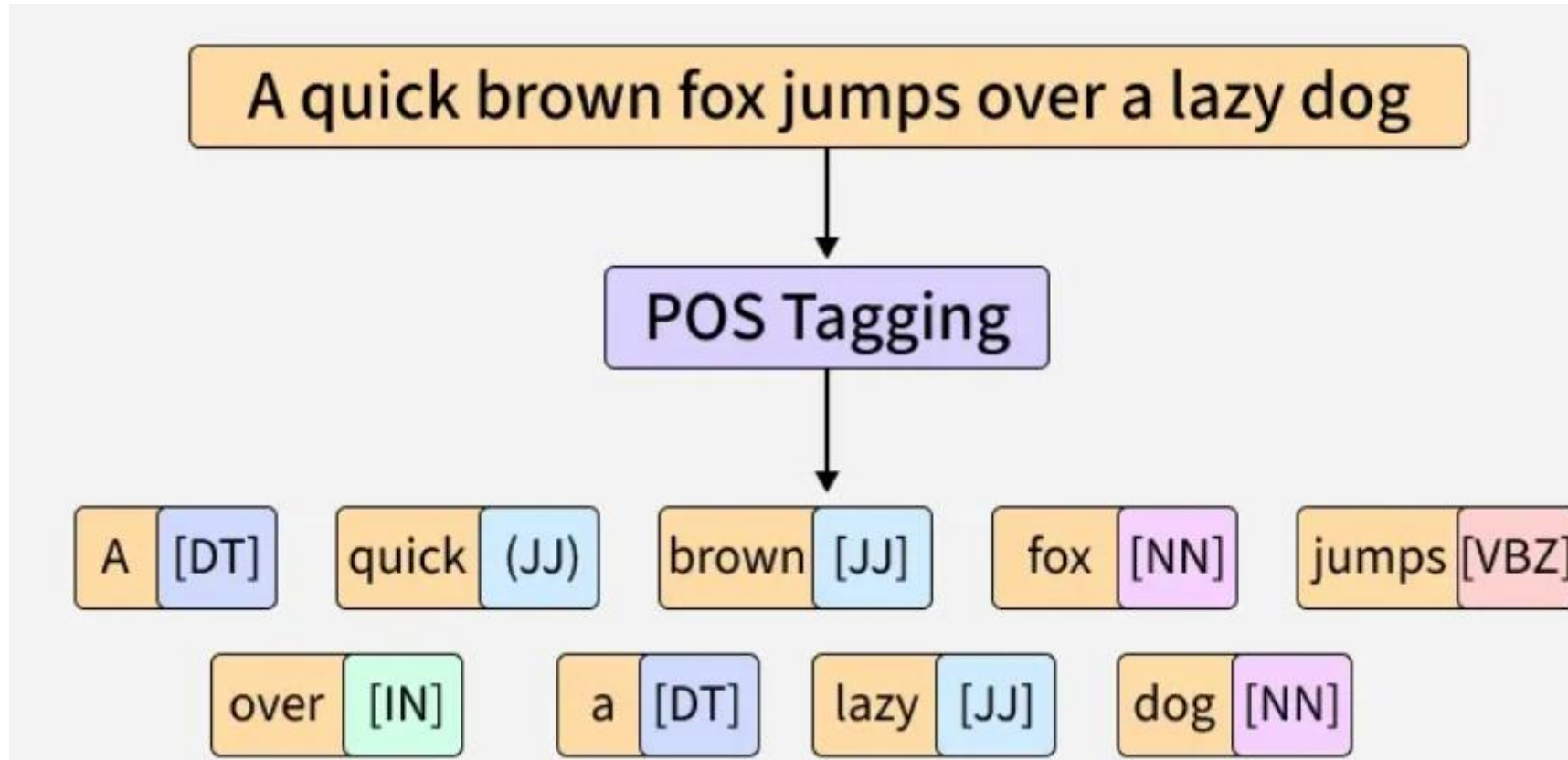
The cat sat on the mat

- The/DET, cat/NOUN, sat/VERB, on/ADP, the/DET, mat/NOUN

Parts-of-speech (also known as POS, word classes, or syntactic categories) are useful because of the large amount of information they give about a word and its neighbours.

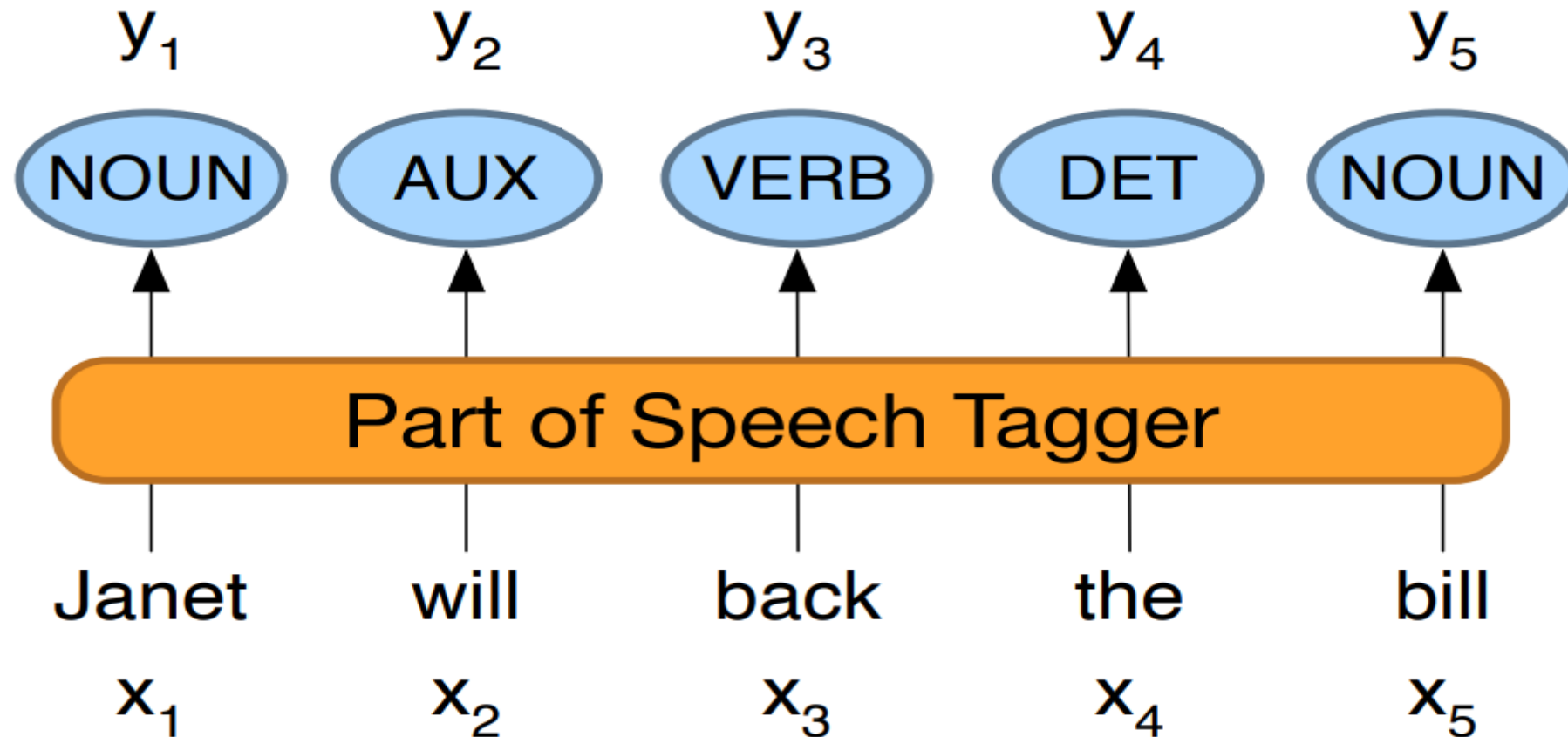
POS tagging is the process of assigning the correct part of speech to each word in a sentence.

Parts-of-Speech Tagging



Parts-of-Speech Tagging

Map from sequence $x_1, \dots, x_n$ of words to $y_1, \dots, y_n$ of POS tags



Parts-of-Speech Tagging

Two classes of words: Open vs. Closed

Parts-of-speech can be divided into two broad categories: ***closed class types*** and ***open class types***.

Closed class words

- Closed classes are those with relatively fixed member ship, such as prepositions—new prepositions are rarely coined.
- Usually *function words*: short, frequent words with grammatical function
 - determiners: a, an, the
 - pronouns: she, he, I
 - prepositions: on, under, over, near, by, ...

Open class words

- Usually content words: Nouns, Verbs, Adjectives, Adverbs
- Plus interjections: oh, ouch, uh-huh, yes, hello
- New nouns and verbs like iPhone or to fax

Open class ("content") words

Nouns

Proper

Janet
Italy

Common

cat, cats
mango

Verbs

Main

eat
went

Adjectives

old green tasty

Adverbs

slowly yesterday

Numbers

122,312
one

Interjections *Ow hello*

... more

Closed class ("function")

Determiners *the some*

Conjunctions *and or*

Pronouns *they its*

Auxiliary

can
had

Prepositions *to with*

Particles *off up*

... more

Parts-of-Speech Tagging

The Penn Treebank Part-of-Speech Tagset

- **The Penn Treebank tagset**, one of the most widely used POS tagging standards in NLP. It was developed for the Penn Treebank project, a large annotated corpus of English.
- The Penn Treebank tagset contains **45 tags**.
- Each tag is a short code (like *NN*, *VBZ*) representing a part of speech and sometimes morphological info (e.g., singular vs. plural, tense, comparative vs. superlative).
- This tagset is the standard for training and evaluating many NLP systems.

Parts-of-Speech Tagging

The Penn Treebank Part-of-Speech Tagset

Tag	Description	Example	Tag	Description	Example
CC	coordin. conjunction	<i>and, but, or</i>	SYM	symbol	<i>+, %, &</i>
CD	cardinal number	<i>one, two</i>	TO	“to”	<i>to</i>
DT	determiner	<i>a, the</i>	UH	interjection	<i>ah, oops</i>
EX	existential ‘there’	<i>there</i>	VB	verb base form	<i>eat</i>
FW	foreign word	<i>mea culpa</i>	VBD	verb past tense	<i>ate</i>
IN	preposition/sub-conj	<i>of, in, by</i>	VBG	verb gerund	<i>eating</i>
JJ	adjective	<i>yellow</i>	VBN	verb past participle	<i>eaten</i>
JJR	adj., comparative	<i>bigger</i>	VBP	verb non-3sg pres	<i>eat</i>
JJS	adj., superlative	<i>wildest</i>	VBZ	verb 3sg pres	<i>eats</i>
LS	list item marker	<i>1, 2, One</i>	WDT	wh-determiner	<i>which, that</i>
MD	modal	<i>can, should</i>	WP	wh-pronoun	<i>what, who</i>
NN	noun, sing. or mass	<i>llama</i>	WP\$	possessive wh-	<i>whose</i>
NNS	noun, plural	<i>llamas</i>	WRB	wh-adverb	<i>how, where</i>
NNP	proper noun, sing.	<i>IBM</i>	\$	dollar sign	<i>\$</i>
NNPS	proper noun, plural	<i>Carolinas</i>	#	pound sign	<i>#</i>
PDT	predeterminer	<i>all, both</i>	“	left quote	<i>‘ or “</i>
POS	possessive ending	<i>’s</i>	”	right quote	<i>’ or ”</i>
PRP	personal pronoun	<i>I, you, he</i>	(	left parenthesis	<i>[, (, {, <</i>
PRP\$	possessive pronoun	<i>your, one’s</i>	)	right parenthesis	<i>],), }, ></i>
RB	adverb	<i>quickly, never</i>	,	comma	<i>,</i>
RBR	adverb, comparative	<i>faster</i>	.	sentence-final punc	<i>. ! ?</i>
RBS	adverb, superlative	<i>fastest</i>	:	mid-sentence punc	<i>: ; ... - -</i>
RP	particle	<i>up, off</i>			

Parts-of-Speech Tagging

The Penn Treebank Part-of-Speech Tagset

Parts-of-speech are generally represented by placing the tag after each word, delimited by a slash, as in the following examples:

- i. The/DT grand/JJ jury/NN commented/VBD on/IN a/DT number/NN of/IN other/JJ topics/NNS ./.
- ii. **There/EX** are/VBP 70/CD children/NNS **there/RB**
- iii. Preliminary/JJ findings/NNS were/VBD **reported/VBN** in/IN today/NN **'s/POS** New/NNP England/NNP Journal/NNP of/IN Medicine/NNP ./.

Parts-of-Speech Tagging

Disambiguation

- Words often have more than one POS.
- For example:
 - Book* can be a **noun** (“I read a *book*”) or a **verb** (“I will *book* a ticket”).
- Context is crucial for determining POS because many words are **ambiguous**.

Parts-of-Speech Tagging

Disambiguation

The amount of tag ambiguity for word types in the Brown and WSJ corpora, from the Treebank-3 (45-tag) tagging.

Types:		WSJ	Brown
Unambiguous	(1 tag)	44,432 (86%)	45,799 (85%)
Ambiguous	(2+ tags)	7,025 (14%)	8,050 (15%)
Tokens:			
Unambiguous	(1 tag)	577,421 (45%)	384,349 (33%)
Ambiguous	(2+ tags)	711,780 (55%)	786,646 (67%)

- **Most word types** (80–86%) are **unambiguous** → they always occur with one tag.
- Example: *Janet* → *NNP*, *funniest* → *JJS*, *hesitantly* → *RB*.
- Only **14–15% of types** are ambiguous, but...

Parts-of-Speech Tagging

Disambiguation

- These ambiguous words are **very frequent** (e.g., can, set, run).
- So in actual text (**tokens**), **55–67%** of word occurrences are ambiguous.
- **WSJ vs. Brown corpus:**
 - WSJ has less ambiguity (55% ambiguous tokens).
 - Brown has more ambiguity (67% ambiguous tokens).
- Reason: **WSJ's narrow financial domain** restricts word usage, while Brown covers diverse genres with wider word usages.
- **Key takeaway:** Tag ambiguity is rare at the vocabulary level, but very common in real running text.

Parts-of-Speech Tagging

Disambiguation

Some of the most ambiguous frequent words are *that*, *back*, *down*, *put* and *set*; here are some examples of the 6 different parts-of-speech for the word *back*:

- earnings growth took a **back/JJ** seat
- a small building in the **back/NN**
- a clear majority of senators **back/VBP** the bill
- Dave began to **back/VB** toward the door
- enable the country to buy **back/RP** about debt
- I was twenty-one **back/RB** then

Parts-of-Speech Tagging

Ambiguity Resolution

Most Frequent Class Baseline: Always compare a classifier against a baseline at least as good as the most frequent class baseline (assigning each token to the class it occurred in most often in the training set).

How good is this baseline? A standard way to measure the performance of part-of-speech taggers is **accuracy**: the percentage of tags correctly labeled on a human labeled test set. If we train on the WSJ training corpus and test on sections 22-24 of the same corpus the most-frequent-tag baseline achieves an accuracy of 92.34%. By contrast, the state of the art in part-of-speech tagging on this dataset is around 97% tag accuracy, a performance that is achievable by number of statistical algorithms including (HMMs, MEMMs, neural networks, rule-based algorithms).

Types of POS Tagging

- **Rule-based approaches** → Use grammar rules and dictionaries.
- **Statistical models / Probabilistic models** → Hidden Markov Model (HMM), Maximum Entropy (MaxEnt) Tagger, Conditional Random Field (CRF), Maximum Entropy Markov Model (MEMM).
- **Transformation-Based Tagging (TBL)** — also called the **Brill Tagger** — is a hybrid POS tagging approach in NLP that combines the simplicity of rule-based methods with the adaptability of statistical learning.
- **Memory-Based Tagging (MBT)** is an approach to POS tagging in NLP that relies on instance-based (lazy) learning, where the model “remembers” all the training examples and decides the tag for a new word based on its similarity to previously seen contexts.
- **Deep learning approaches** → BiLSTM, Transformers (e.g., BERT).

Markov Property

In probability theory and statistics, the term Markov property refers to the memoryless property of a stochastic process, which means that its future evolution is independent of its history, it only depends on the current state. It is named after the Russian mathematician Andrey Markov.

Markov Model

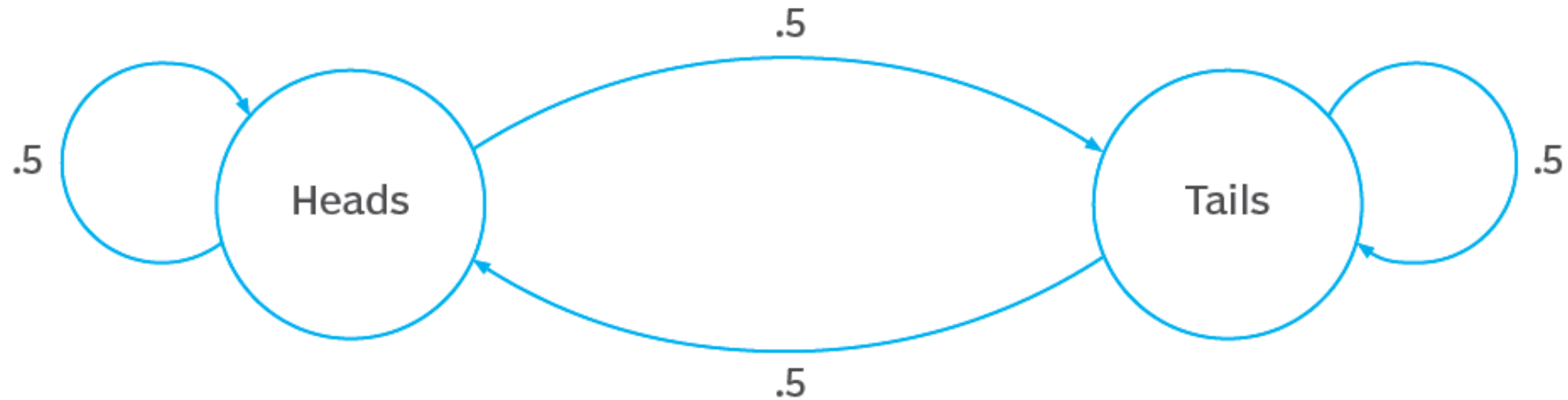
A model that follows the Markov property.

Markov models

	System state is fully observable	System state is partially observable
System is autonomous	Markov chain	Hidden Markov model
System is controlled	Markov decision process	Partially observable Markov decision process

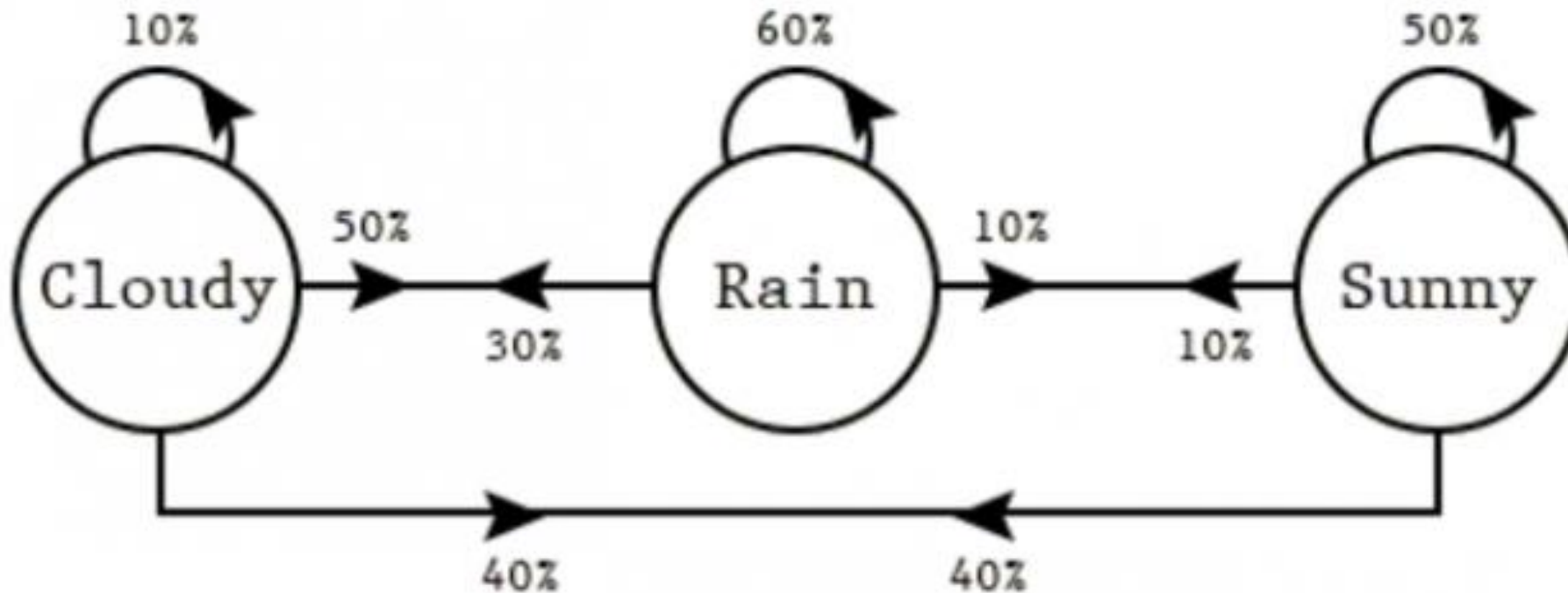
Markov Chain Example

Markov chain coin toss



Markov Chain Example

What is the probability that Wednesday will be cloudy if Monday is sunny?



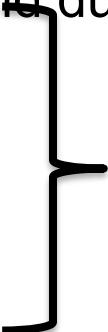
Hidden Markov Model (HMM)

Markov chain with unobservable/hidden states.

Example 1:

Lea is our imaginary friend and during the day she does either of these four things:

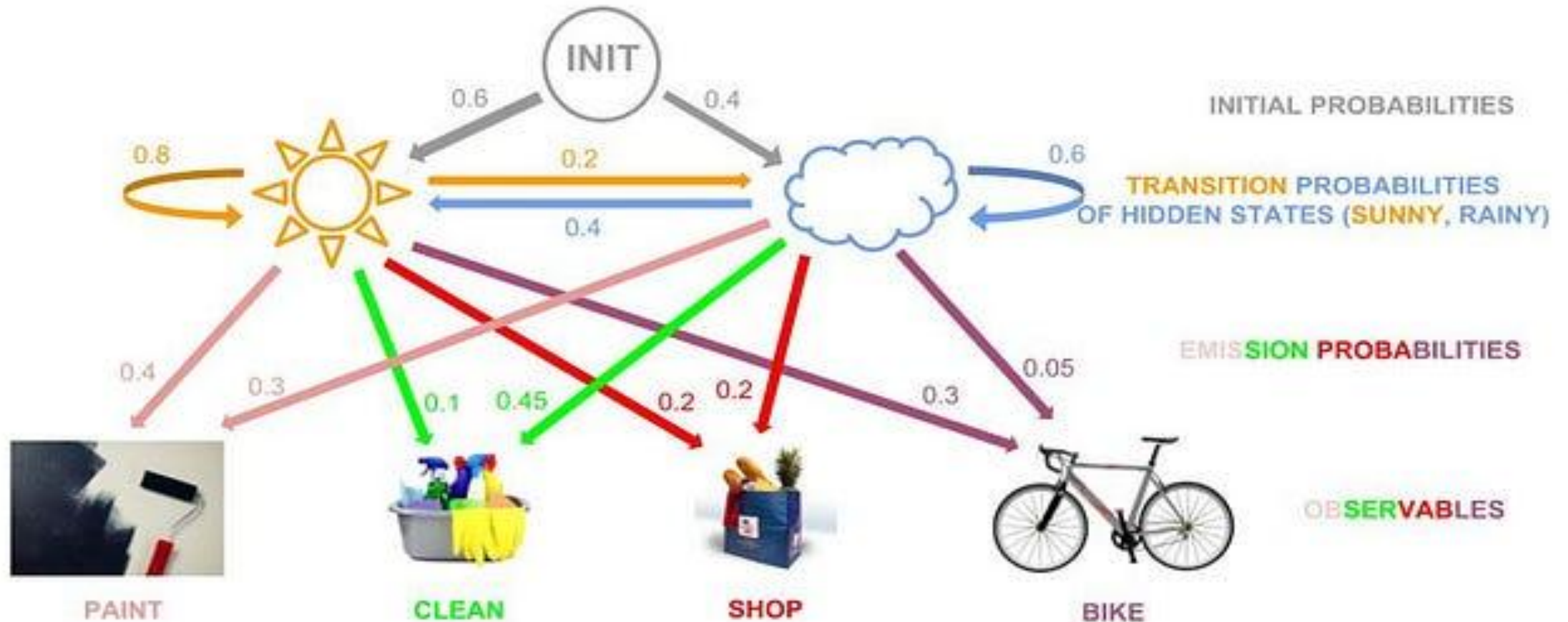
- Painting
- Cleaning the house
- Biking
- Shopping for groceries



Observable States

From Lea's activities, we want to know whether the day has been sunny or rainy. These two are going to be our **hidden states**.

Hidden Markov Model (HMM)



Hidden Markov Model (HMM)

To start off, a Hidden Markov Model consists of the following properties:

Hidden States S : in the example above the hidden states are Sunny and Rainy, and they get grouped into a set S .

Observables O : Paint, Clean, Shop and Bike. They get grouped into a set O .

Initial Probabilities π : a matrix of the initial likelihood of the state at time $t = 0$. In this case the likelihood that it is Sunny on the first day is 0.6, while the likelihood that it is Rainy is 0.4.

$$\pi = [0.6, 0.4]$$

Note: every row of the following matrices must add up to 1 since they represent a probability.

Hidden Markov Model (HMM)

Transition Probabilities A: a matrix that represents the probability of transitioning to another state given the current state.

For example, if the **current state is Sunny** the probability that the **day after** is **Sunny** as well is **0.8**, whereas the probability that the day after is **Rainy** is **0.2**.

Similarly if **today is Rainy**, the probability that **tomorrow** is **Rainy** as well is **0.6**, while the probability that **tomorrow** is **Sunny** is **0.4**.

Hidden Markov Model (HMM)

Transition Probabilities A

$$A =$$

		to	
from			
		0.8	0.2
		0.4	0.6

Hidden Markov Model (HMM)

Emission Probabilities B: a matrix that represents the probability of seeing a specific observable given a hidden state.

For example, the probability of **Clean** on a **Sunny day** is **0.1**, whereas the probability of **Clean** on a **Rainy day** is **0.45**.

$B =$

		PAINT	CLEAN	SHOP	BIKE
from		0.4	0.1	0.2	0.3
		0.3	0.45	0.2	0.05

to

Hidden Markov Model (HMM)

Model's Properties

$$S = \{ S_{\text{sunny}}, S_{\text{rainy}} \} \quad (\text{Hidden States})$$

$$O = \{ O_{\text{clean}}, O_{\text{bike}}, O_{\text{shop}}, O_{\text{paint}} \} \quad (\text{Observables})$$

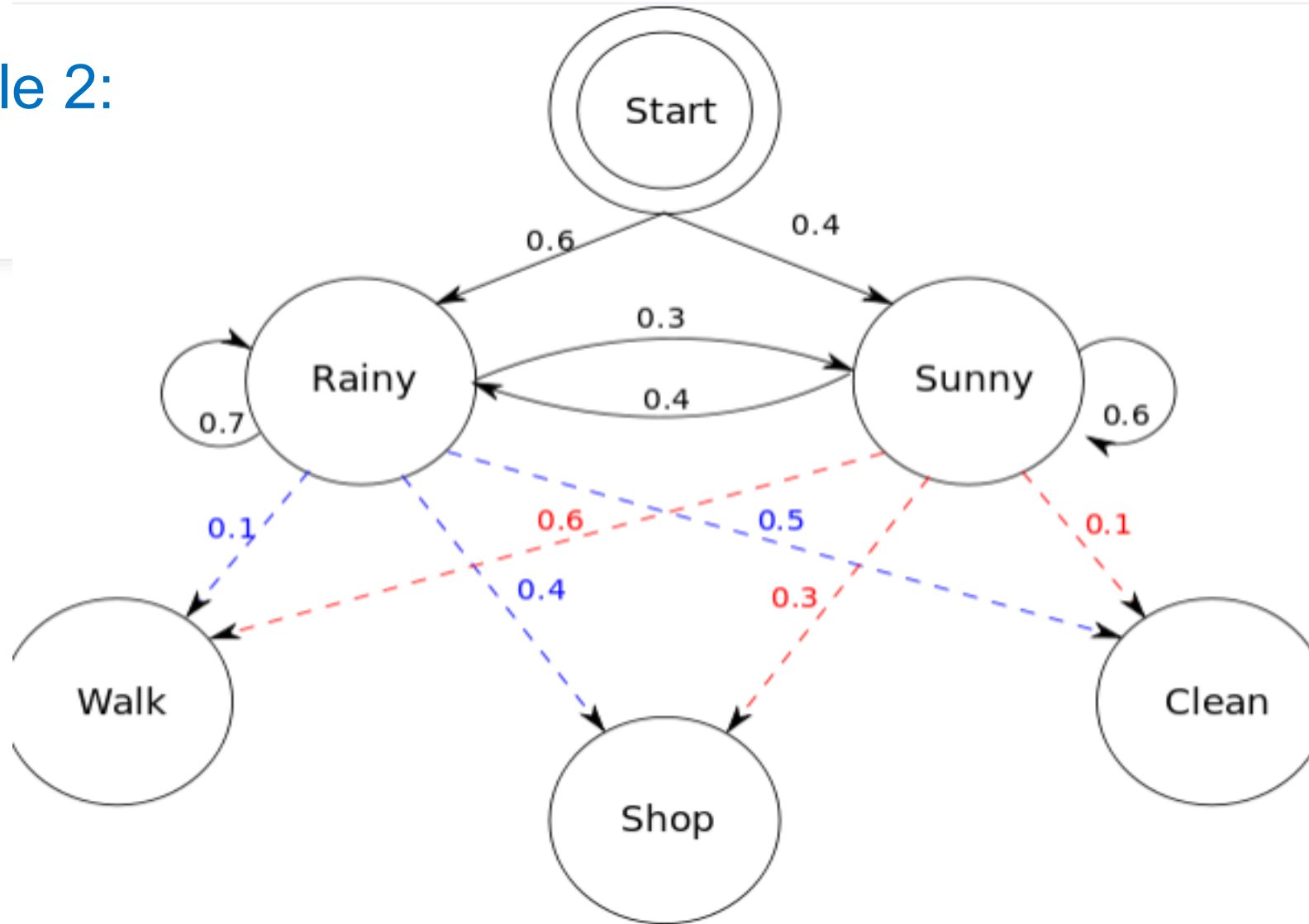
$$\pi = [0.6 \quad 0.4] \quad (\text{Initial Probabilities})$$

$$A = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \quad (\text{Transition Probabilities})$$

$$B = \begin{bmatrix} 0.4 & 0.1 & 0.2 & 0.3 \\ 0.3 & 0.45 & 0.2 & 0.05 \end{bmatrix} \quad (\text{Emission Probabilities})$$

Hidden Markov Model (HMM)

Example 2:



Hidden Markov Model (HMM)

```
states = ('Rainy', 'Sunny')

observations = ('walk', 'shop', 'clean')

start_probability = {'Rainy': 0.6, 'Sunny': 0.4}

transition_probability = {
    'Rainy' : {'Rainy': 0.7, 'Sunny': 0.3},
    'Sunny' : {'Rainy': 0.4, 'Sunny': 0.6},
}

emission_probability = {
    'Rainy' : {'walk': 0.1, 'shop': 0.4, 'clean': 0.5},
    'Sunny' : {'walk': 0.6, 'shop': 0.3, 'clean': 0.1},
}
```

References

[1] https://setosa.io/ev/markov-chains/?utm_source=chatgpt.com

[2] https://medium.com/%40Ayra_Lux/hidden-markov-models-part-1-the-likelihood-problem-8dd1066a784e

HMM

Markov Chain Definition: A model for probabilities of sequences of random variables (states).

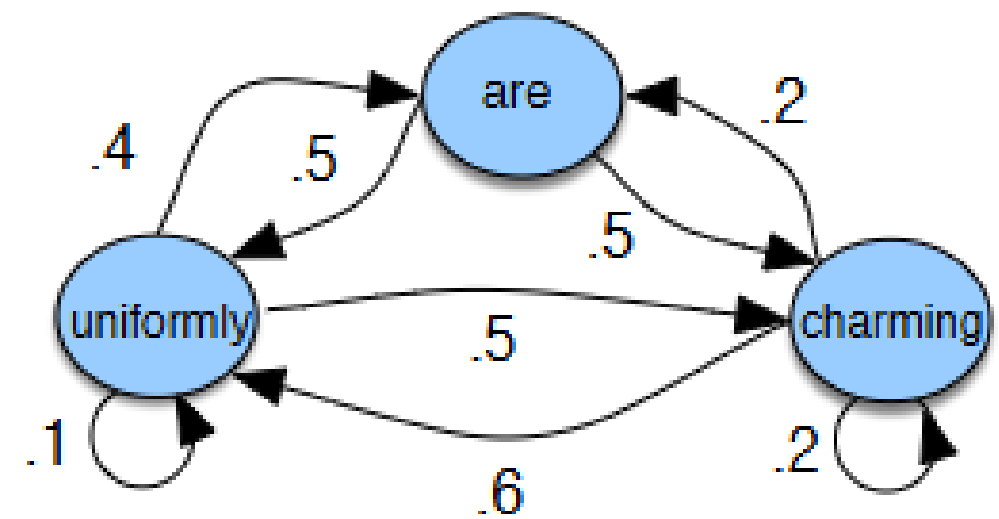
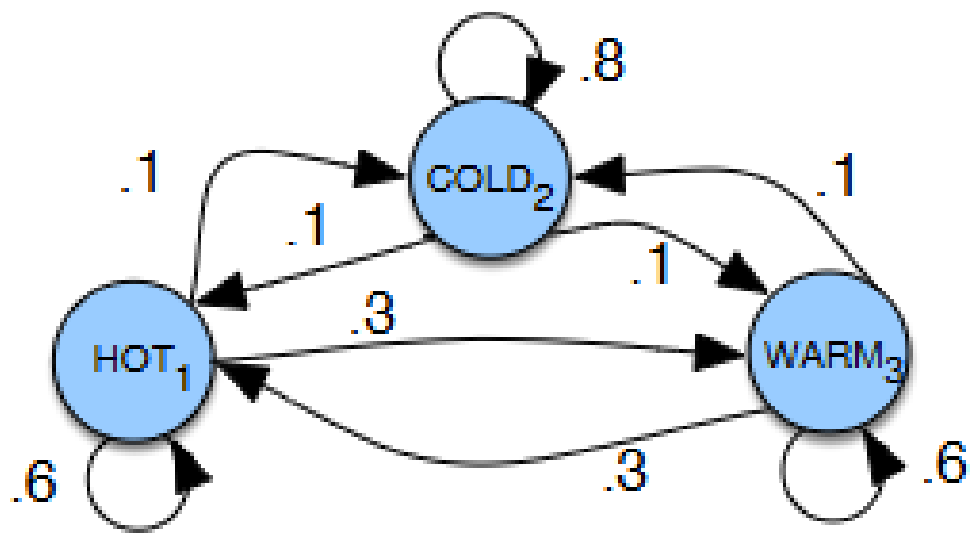
State Values: Each state can take values from a set, these sets can be words, or tags, or symbols representing anything, like the weather.

Key Assumption: Future depends only on the current state, not on past states.

Implication: To predict the next state, only the current state matters; prior states influence the future indirectly via the current state.

Example: It's as if to predict tomorrow's weather you could examine today's weather but you weren't allowed to look at yesterday's weather.

HMM Relation: [Hidden Markov Models \(HMMs\)](#) are based on augmenting Markov chains.



Formally, a Markov chain is specified by the following components:

$$Q = q_1 q_2 \dots q_N$$

a set of N states

$$A = a_{11} a_{12} \dots a_{N1} \dots a_{NN}$$

a **transition probability matrix** A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^n a_{ij} = 1 \quad \forall i$

$$\pi = \pi_1, \pi_2, \dots, \pi_N$$

an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^N \pi_i = 1$

HMM in POS Tagging

- **Markov Chain Use:** Computes probabilities for sequences of **observable events**.
- **Hidden Events:** Sometimes, the events of interest are **not directly observable**.

Example: **Part-of-speech (POS) tags** are hidden; we only see the words.

- **Hidden Markov Model (HMM):**
 - Models **both observed events** (e.g., **words**) and **hidden events** (e.g., **POS tags**).
 - Hidden events are considered **causal factors** in the probabilistic model.

HMM in POS Tagging

We don't normally observe part-of-speech tags in a text. Rather, we see words, and must infer the tags from the word sequence. We call the tags hidden because they are not observed.

An HMM is specified by the following components:

$Q = q_1 q_2 \dots q_N$	a set of N states
$A = a_{11} \dots a_{ij} \dots a_{NN}$	a transition probability matrix A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$
$O = o_1 o_2 \dots o_T$	a sequence of T observations , each one drawn from a vocabulary $V = v_1, v_2, \dots, v_V$
$B = b_i(o_t)$	a sequence of observation likelihoods , also called emission probabilities , each expressing the probability of an observation o_t being generated from a state q_i
$\pi = \pi_1, \pi_2, \dots, \pi_N$	an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$

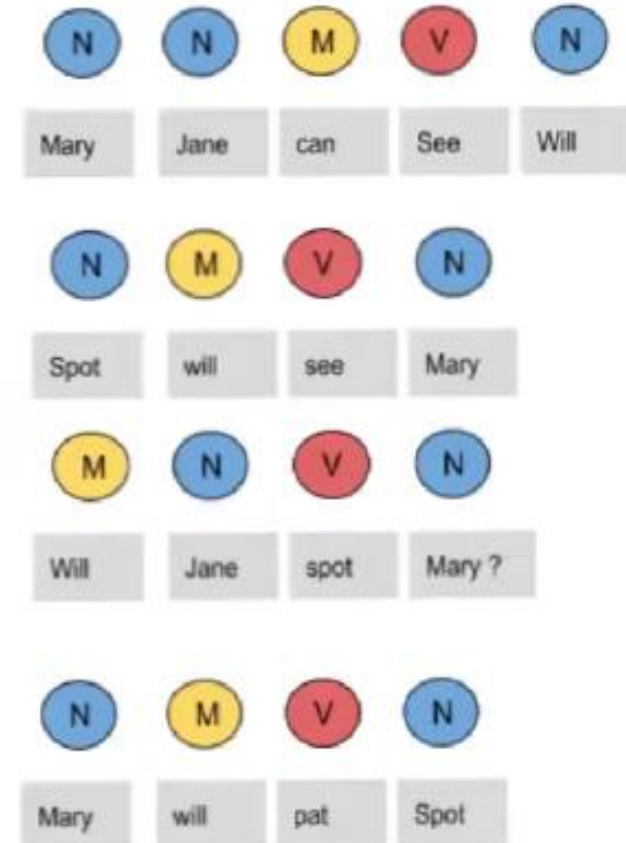
Example

Let us calculate the above two probabilities for the set of sentences below

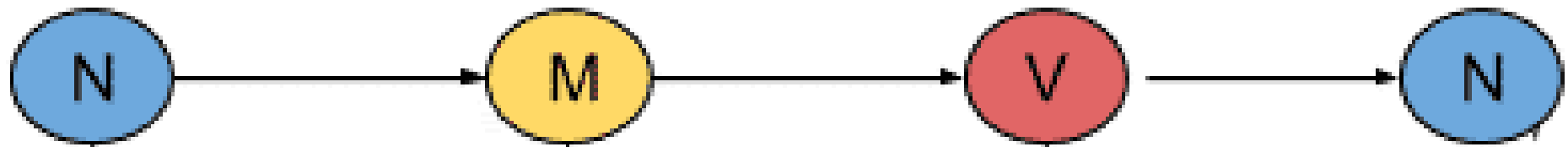
Mary Jane can see Will
Spot will see Mary
Will Jane spot Mary?
Mary will pat Spot

Note that Mary Jane, Spot, and Will are all names.

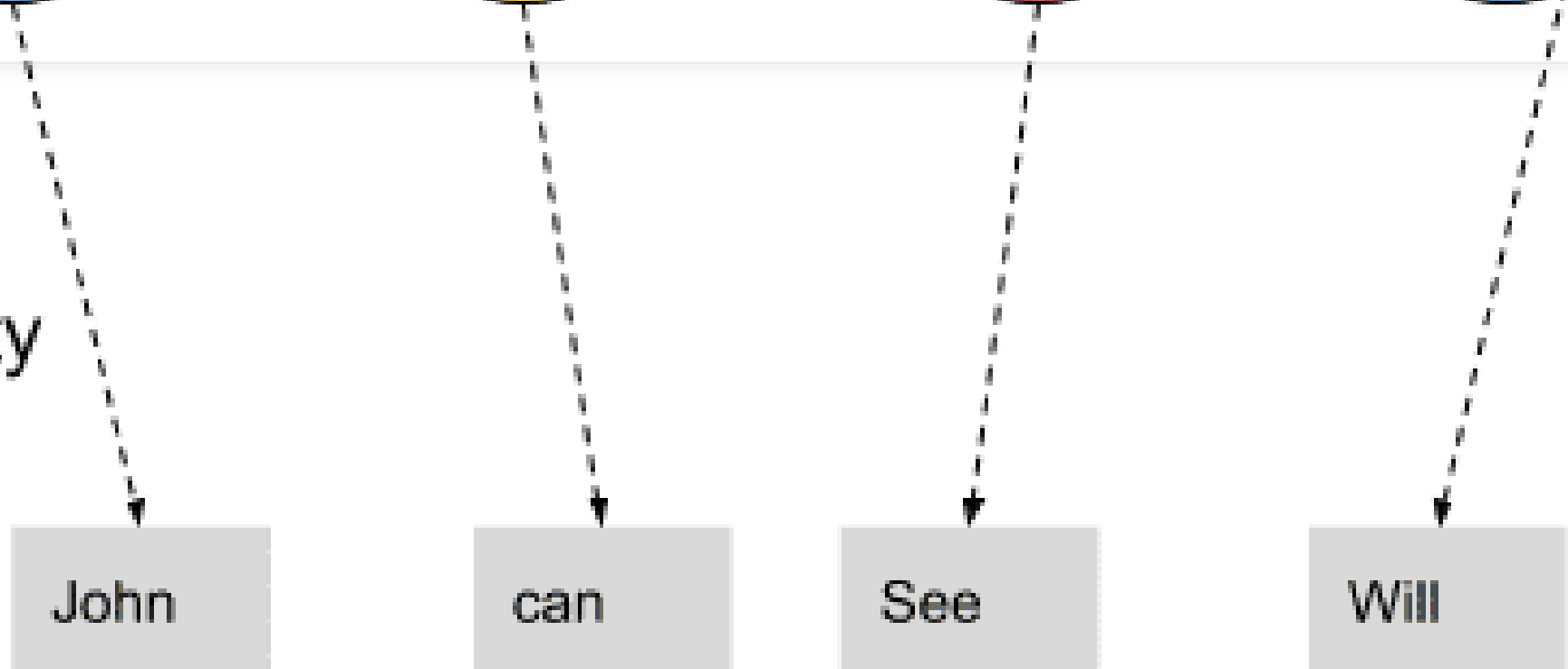
Take a new sentence and tag them with wrong tags.
Let the sentence, '**Will can spot Mary**' be tagged as-



Transition Probability



Emission Probability



N

N

M

V

N

Mary

Jane

can

See

Will

N

M

V

N

Spot

will

see

Mary

M

N

V

N

Will

Jane

spot

Mary ?

N

M

V

N

Mary

will

pat

Spot

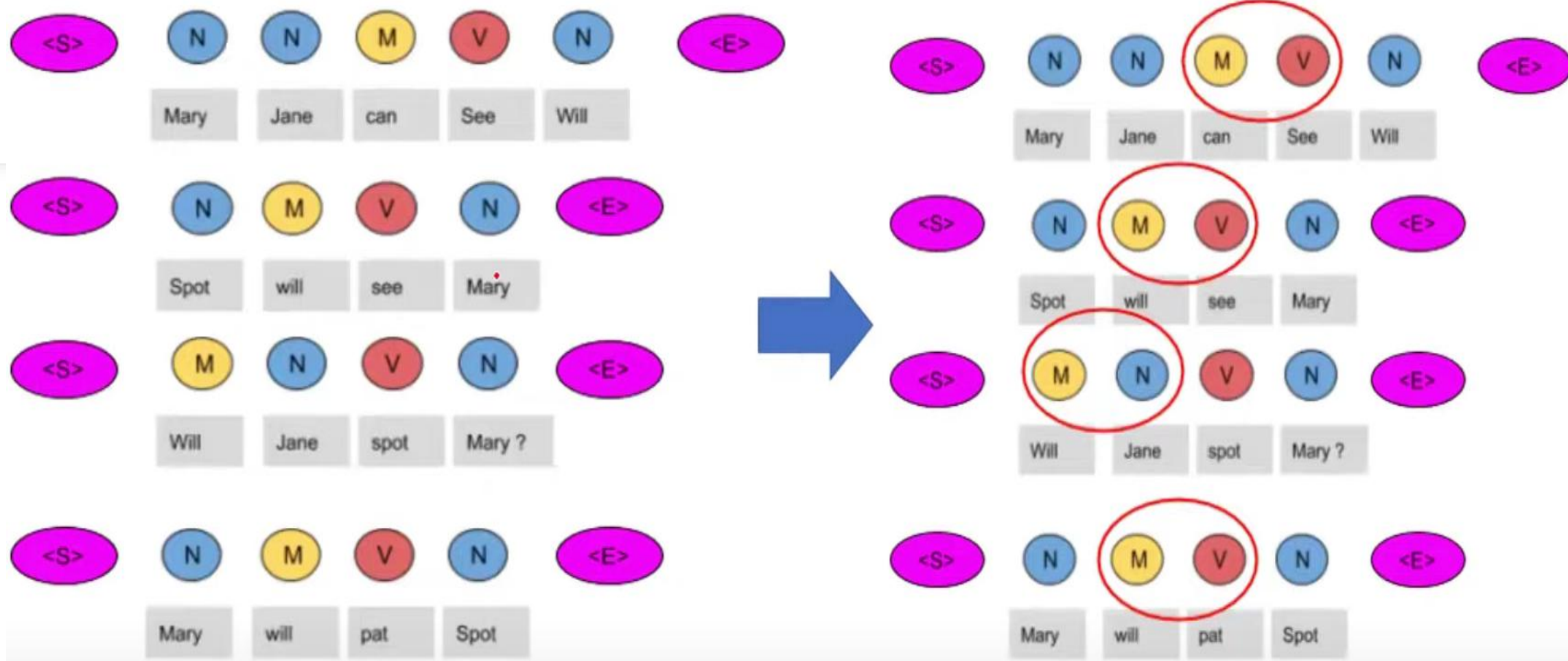
To calculate the emission probabilities, let us create a counting table.

Words	Noun	Model	Verb
Mary	4	0	0
Jane	2	0	0
Will	1	3	0
Spot	2	0	1
Can	0	1	0
See	0	0	2
pat	0	0	1

emission probabilities

Words	Noun	Model	Verb
Mary	$4/9$	0	0
Jane	$2/9$	0	0
Will	$1/9$	$3/4$	0
Spot	$2/9$	0	$1/4$
Can	0	$1/4$	0
See	0	0	$2/4$
pat	0	0	$1/4$

Transition Probabilities



	N	M	V	<E>
<S>	3	1	0	0
N	1	3	1	4
M	1	0	3	0
V	4	0	0	0

Transition Probabilities

	N	M	V	<E>
<S>	3/4	1/4	0	0
N	1/9	3/9	1/9	4/9
M	1/4	0	3/4	0
V	4/4	0	0	0

Take a new sentence and tag them with wrong tags.
Let the sentence, '**Will can spot Mary**' be tagged as-

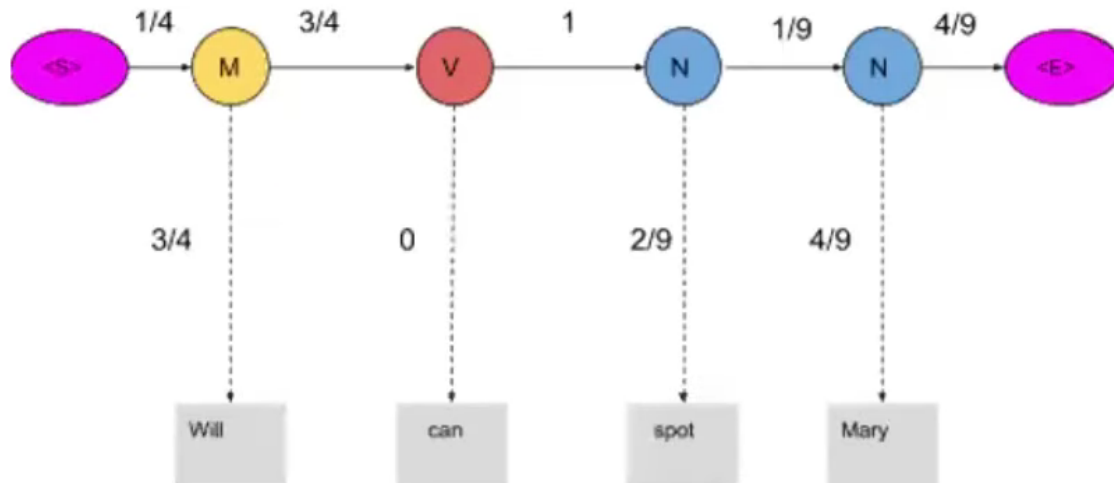
Will as a model

Can as a verb

Spot as a noun

Mary as a noun

Now calculate the probability of this sequence being correct in the following manner.



$$1/4 * 3/4 * 3/4 * 0 * 1 * 2/9 * 1/9 * 4/9 * 4/9 = 0$$



1/9

1/4

1/4

4/9



$$\frac{3}{4} \times \frac{1}{9} \times \frac{3}{9} \times \frac{1}{4} \times \frac{3}{4} \times \frac{1}{4} \times 1 \times \frac{4}{9} \times \frac{4}{9} = 0.00025720164$$

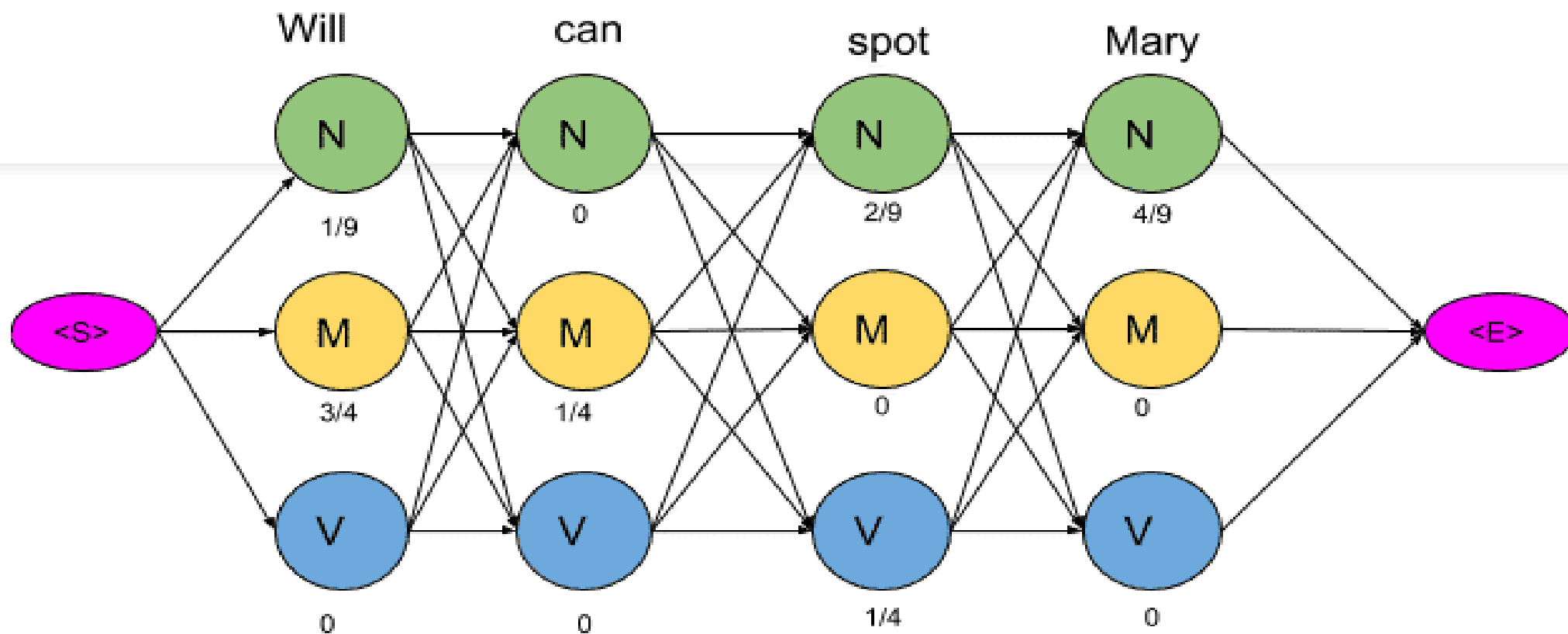
Will

can

spot

Mary

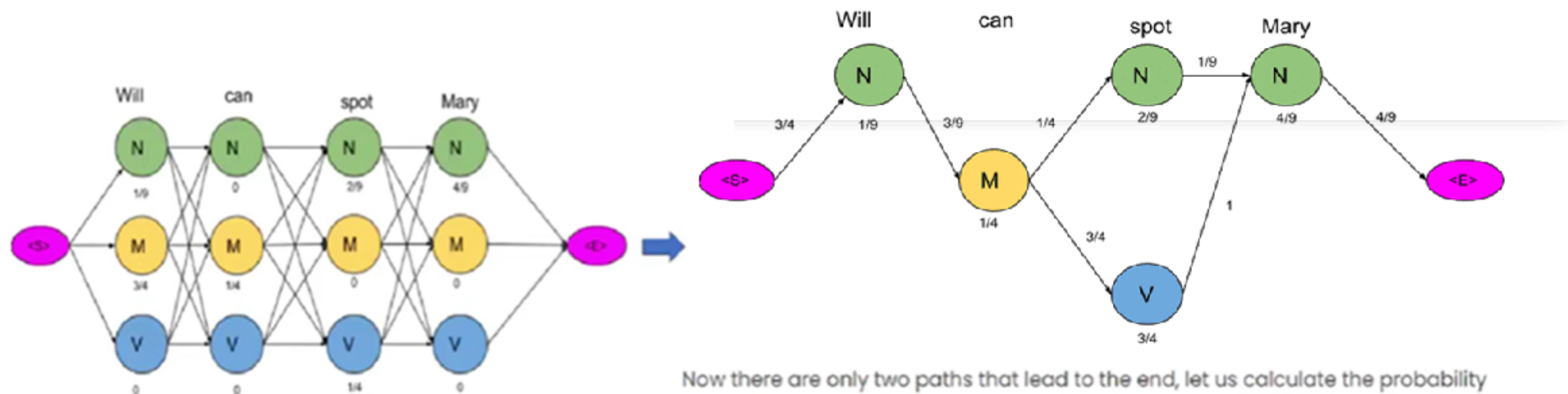
All possible combinations with three tags(N, M and V)



$3^4 = 81$ Total possibilities (different combinations of tags are possible)

- Keeping into consideration just three POS tags we have mentioned, 81 different combinations of tags can be formed.
- Number of tags used :3
- Number of words to tag : 4 ('Will can spot Mary')
- Total possibility : $3^4 = 81$
- In this case, calculating the probabilities of all 81 combinations seems achievable.
- But when the task is to tag a larger sentence and all the POS tags in the Penn Treebank project are taken into consideration, the number of possible combinations grows exponentially and this task seems impossible to achieve.

- The next step is to **delete all the vertices and edges with probability zero**, also the vertices which do not lead to the endpoint are removed.



Clearly, the probability of the second sequence is much higher and hence the HMM is going to tag each word in the sentence according to this sequence.

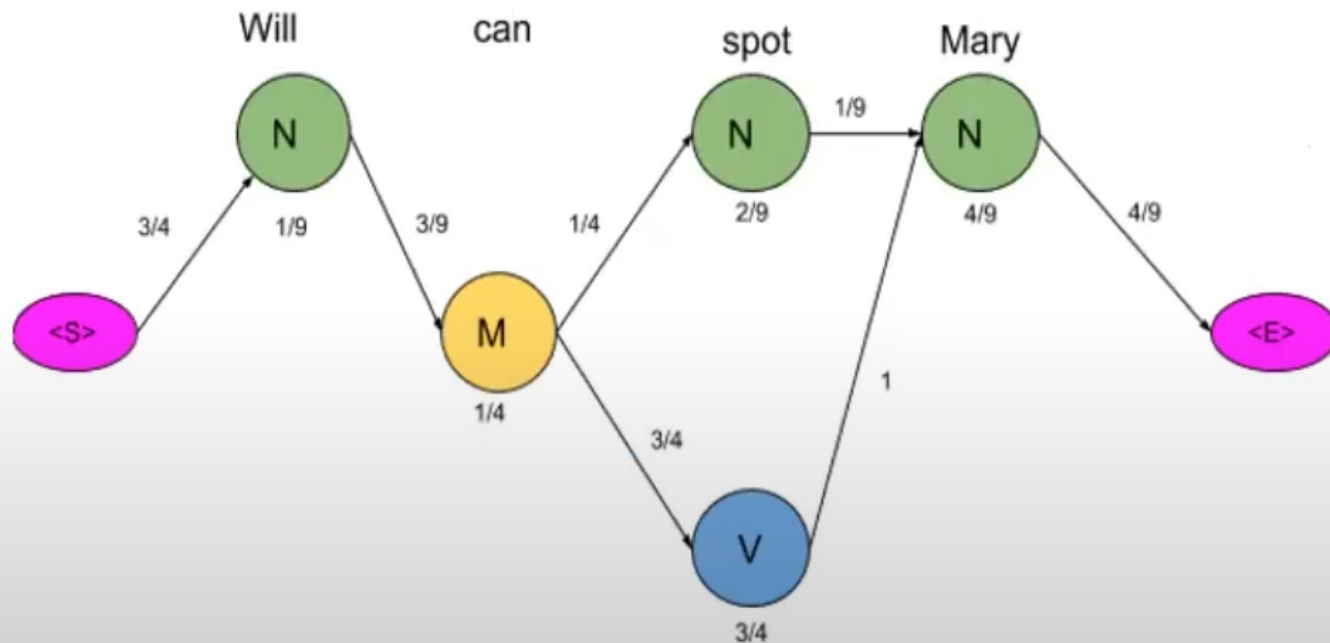
Now there are only two paths that lead to the end, let us calculate the probability associated with each path.

$$\langle S \rangle \rightarrow N \rightarrow M \rightarrow N \rightarrow N \rightarrow \langle E \rangle = 3/4 * 1/9 * 3/9 * 1/4 * 1/4 * 2/9 * 1/9 * 4/9 * 4/9 = 0.00000846754$$

$$\langle S \rangle \rightarrow N \rightarrow M \rightarrow V \rightarrow N \rightarrow \langle E \rangle = 3/4 * 1/9 * 3/9 * 1/4 * 3/4 * 1/4 * 1/4 * 4/9 * 4/9 = 0.00025720164$$

Removal of unwanted links and nodes

The next step is to delete all the vertices and edges with probability zero, also the vertices which do not lead to the endpoint are removed.

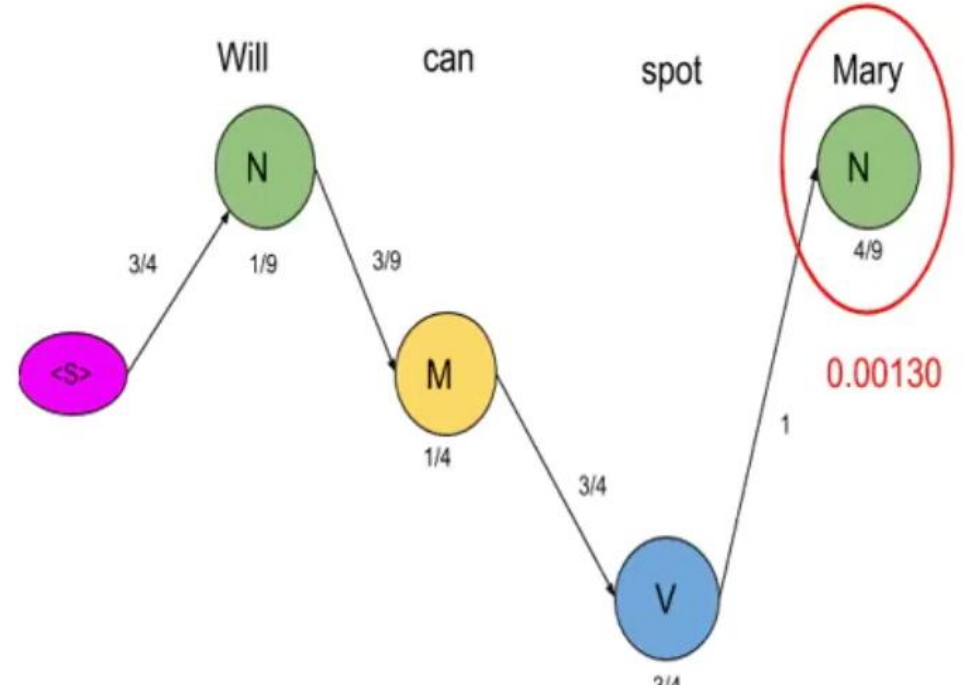
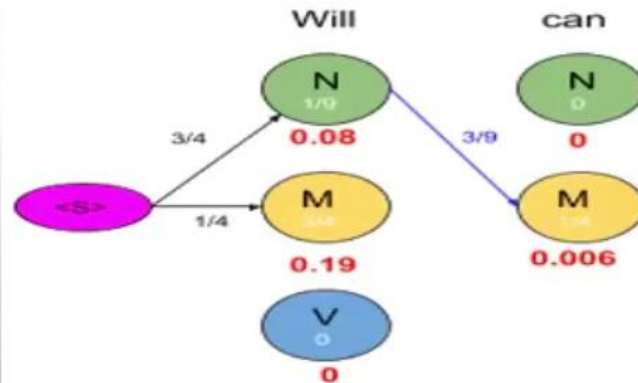
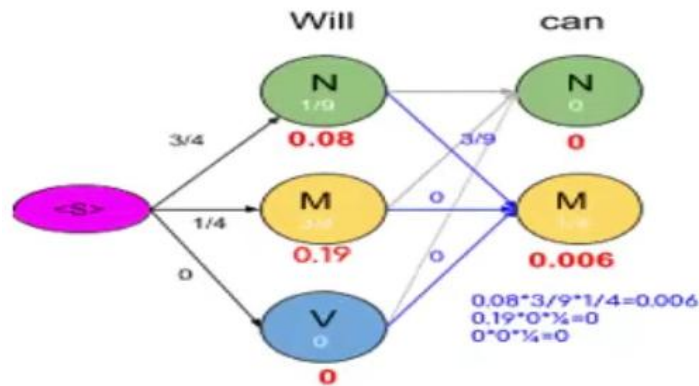
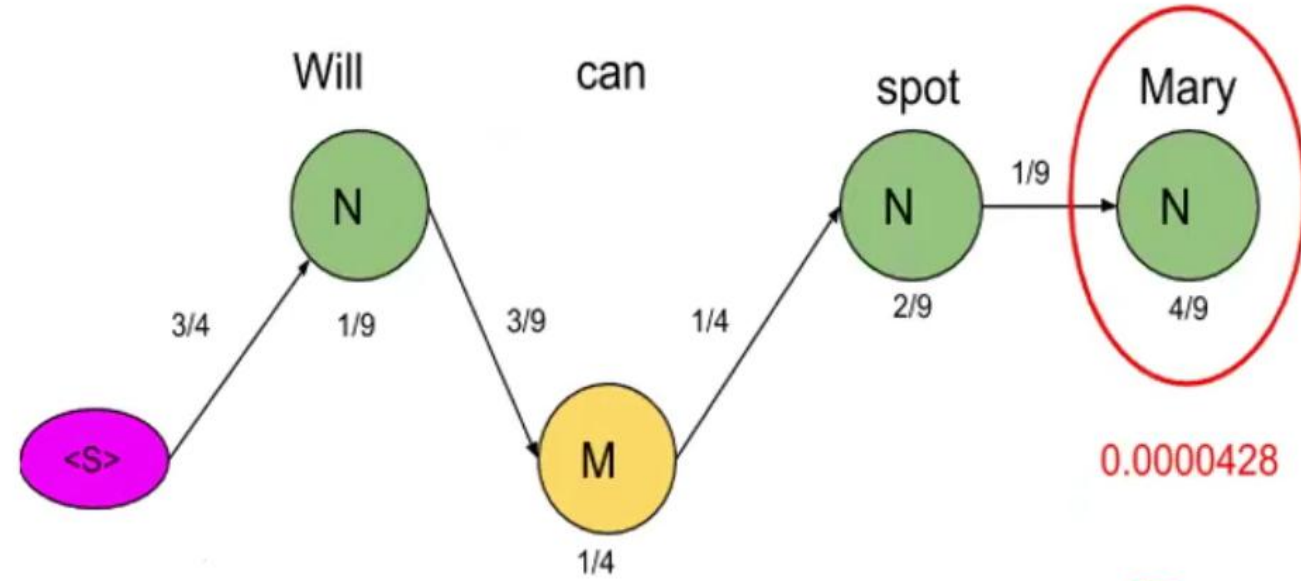
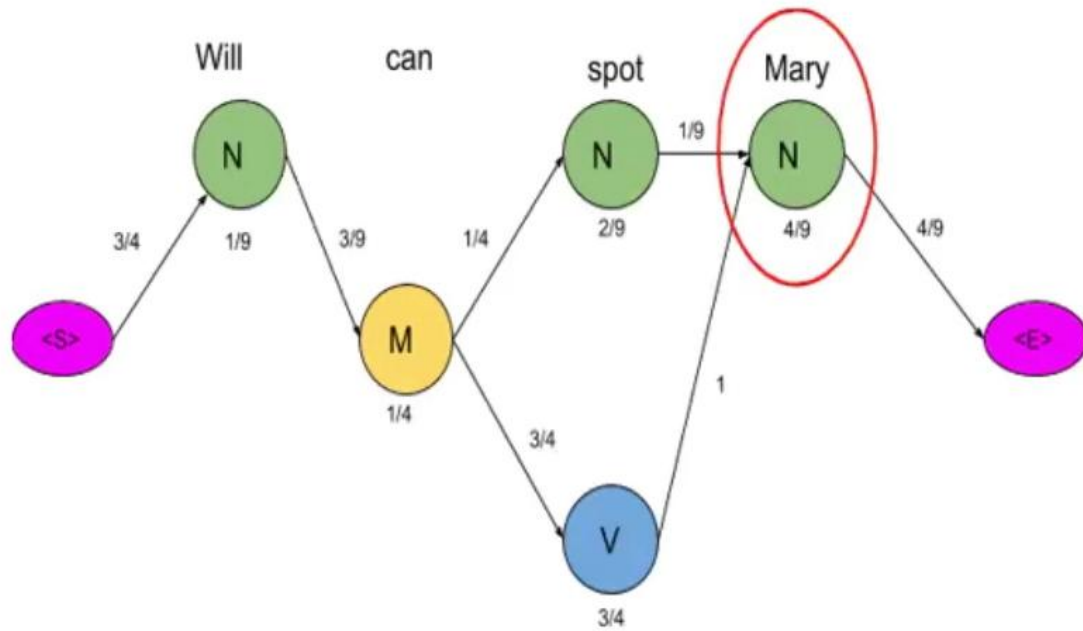


Clearly, the probability of the second sequence is much higher and hence the HMM is going to tag each word in the sentence according to this sequence.

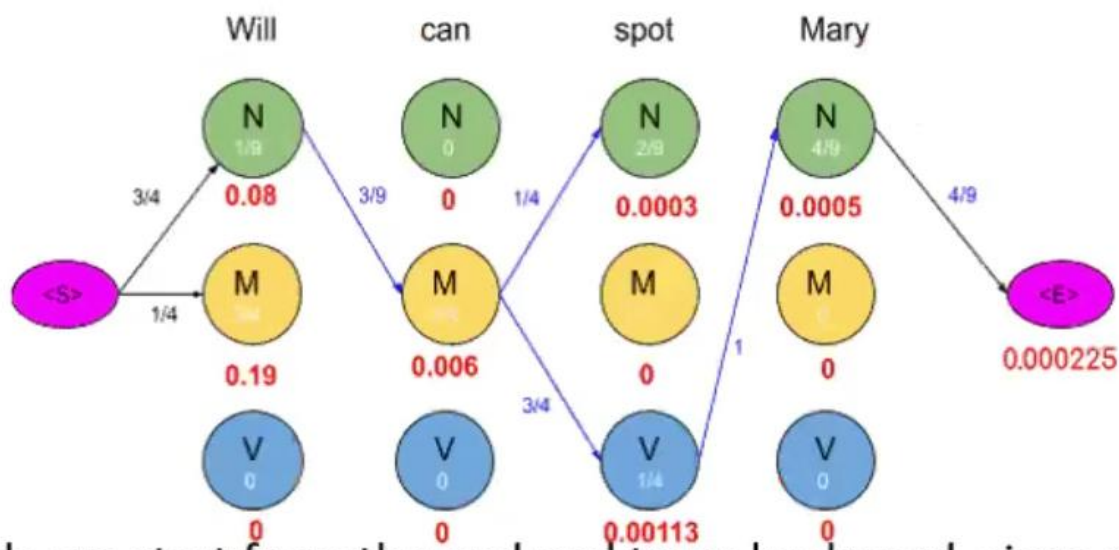
$$\langle S \rangle \rightarrow N \rightarrow M \rightarrow N \rightarrow N \rightarrow \langle E \rangle = 3/4 * 1/9 * 3/9 * 1/4 * 1/4 * 2/9 * 1/9 * 4/9 * 4/9 = 0.00000846754$$

$$\langle S \rangle \rightarrow N \rightarrow M \rightarrow N \rightarrow V \rightarrow \langle E \rangle = 3/4 * 1/9 * 3/9 * 1/4 * 3/4 * 1/4 * 1 * 4/9 * 4/9 = 0.00025720164$$

Optimizing HMM with Viterbi Algorithm

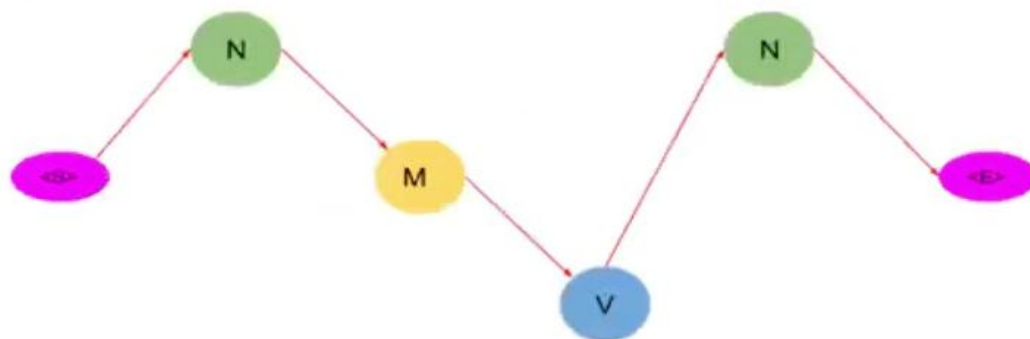


- the probabilities of all paths leading to a node are calculated and we remove the edges or path which has lower probability cost.
- Also, you may notice some nodes having the probability of zero and such nodes have no edges attached to them as all the paths are having zero probability.
- The graph obtained after computing probabilities of all paths leading to a node is shown below:



To get an optimal path, we start from the end and trace backward, since each state has only one incoming edge,

This gives us a path as shown below



Example

Let's now work through an example of computing the best sequence of tags that corresponds to the following sequence of words.

- Janet will back the bill

The correct series of tags is:

- Janet/NNP will/MD back/VB the/DT bill/NN

The below table lists the a_{ij} probabilities for transitioning between the hidden states (part-of-speech tags).

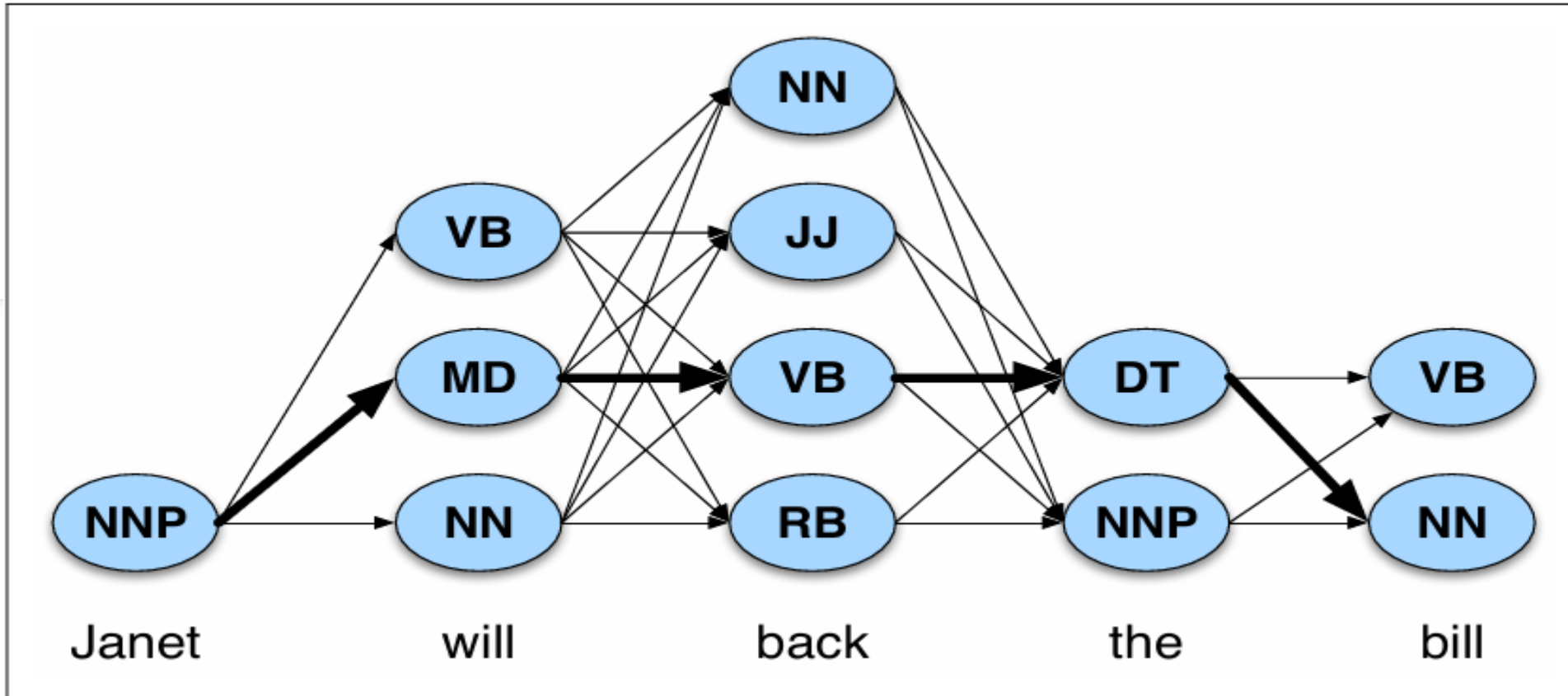
	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB|MD)$ is 0.7968.

Expresses the $b_i(o_t)$ probabilities, the observation likelihoods of words given tags.

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Observation likelihoods B computed from the WSJ corpus without smoothing.



A schematic of the tagging task for the sample sentence, showing the ambiguities for each word and the correct tag sequence as the highlighted path through the hidden states.

Morphology

Morphology studies the internal structure of words, how words are built up from smaller meaningful units called **morphemes**.

Morpheme: It is minimal meaning bearing unit in a language.

Examples:

- dogs: 2 morphemes- 'dog' and 's'.
 's' is a plural marker on nouns.
- Unladylike: 3 morphemes
 un- 'not'
 lady- 'well-behaved woman'
 like- 'having the characteristic of' ns.
- fox: 1 morpheme (singular); foxes: 2 morphemes (plural)

Morphology

Classes of Morphemes

Two broad classes of morphemes are:

- **Stems** – the core meaningful units, the root of the word.
- **Affixes** – bits and pieces adhering to stems to change their meanings and grammatical functions.
 - **Prefixes** – precede the stem: do / undo
 - **Suffixes** – follow the stem: eat / eats
 - **Infixes** – are inserted inside the stem
 - **Circumfixes** – precede and follow the stem

Morphology

Affixes – Prefix vs Suffix

PREFIX

Word or part of word added before stem or root. **Examples:**

- bi (two): bicycle, bilingual
- dis (not): dishonest, disagree, disapprove, disconnect
- in (not, without, lacking): injustice, incomplete, invisible
- ir (not, non, opposite) :irreversible, irreplaceable, irregular
- More common prefix example

SUFFIX

Word or part of word added after stem or root. **Examples:**

- able (can be done): preventable, adaptable, predictable, credible
- er (some one who performs an action): teacher, helper, preacher, dancer
- -ion (the action or process of): celebration, opinion, decision, revision
- More common suffix example

Morphology

Affixes – Infix vs Circumfixes

INFIX

Word or part of word added in between the stem or root word.

Examples:

- passersby: plural of passerby
- mothers-in-law : plural of mother-in-law
- More common in informal English.

CIRCUMFIXES

Circumfixes attach to both the beginning and the end of a word

Examples:

- in-adequate-ness
- in-correct-ness
- im-perfect-ion
- less common in English.

Morphology

Affix Types

Affix type	How it attaches to stem
Prefix	X-STEM
Suffix	STEM-X
Infix	ST-X-EM
Circumfix	X-STEM-X

Morphology

There are two broad (and partially overlapping) classes of morphology

- 1) Inflectional Morphology
- 2) Derivational Morphology

INFLECTIONAL MORPHOLOGY

Grammatical: Add the grammatical property to the word like tense, number (plural), case, gender, comparison

Creates new forms of the same word: bring, brought, brings, bringing.

DERIVATIONAL MORPHOLOGY

Creates new words by changing part-of-speech: logic, logical, illogical, illogicality, logician

Fairly systematic but some derivations missing: sincere - sincerity, scarce - scarcity, curious - curiosity, fierce - fierceness

Morphology

Inflectional Morphology

- Inflectional morphology deals with **grammatical variations of the same word**. It changes the form of a word **without changing its core meaning or category** (noun stays noun, verb stays verb).
- It shows **tense, number, gender, case, aspect, or degree**.

Examples:

- Noun Inflection:
 - **cat** → **cats** (plural)
- Verb Inflection:
 - **walk** → **walked** (past tense)
 - **walk** → **walking** (progressive aspect)
- Adjective Inflection:
 - **fast** → **faster** → **fastest** (comparative and superlative)
- Inflectional analysis is crucial for **lemmatization** (finding the dictionary/base form of words) and **POS tagging**, where the system must recognize that *walked* and *walking* are forms of *walk*.

Morphology

Derivational Morphology

- Derivational morphology creates **new words** by adding morphemes (prefixes/suffixes) that often **change the meaning and sometimes the grammatical category** of the base word.
- Unlike inflection, derivation can form **completely new lexical items**.

Examples:

- Noun → Adjective:
 - **nation** → **national**
- Verb → Noun:
 - **teach** → **teacher**
- Adjective → Adverb:
 - **happy** → **happily**
- Prefix changes meaning:
 - **possible** → **impossible**

Morphology

Inflectional Morphology vs Derivational Morphology

INFLECTION

- 1 An *inflectional morpheme* is added to a noun, verb, adjective or adverb to assign a particular **grammatical property** to that word such as: tense, number, possession, or comparison.

- Cat $\Rightarrow$ Cats, Teach $\Rightarrow$ Teaches
- Clean $\Rightarrow$ Cleaned, Pretty $\Rightarrow$ Prettier

DERIVATIONAL

- 1 *Derivational morphemes* form a new words that differ either in grammatical category (parts-of-speech) or in meaning from their bases.

- **Change in Meaning:** Leaf $\Rightarrow$ leaflet, Pure $\Rightarrow$ Impure
- **Change in Grammatical Category:** Help (verb) $\Rightarrow$ Helper (noun), Logic (noun) $\Rightarrow$ Logical (adjective)

Morphology

Inflectional Morphology vs Derivational Morphology

INFLECTION

- 2 These do not change the essential meaning or the grammatical category (Parts-of-speech) of a word. Adjectives stay adjectives, nouns remain nouns, and verbs stay verbs.
- Clean (Verb) $\Rightarrow$ cleaned (Verb)
 - Cat (Noun) $\Rightarrow$ Cats (Noun)

DERIVATIONAL

- 2 *Derivational morphemes* tend to change the grammatical category of a word **but not always!**
- ful like in 'beautiful' $\Rightarrow$ beauty (N) + ful (A) = beautiful (A)
 - able like in 'moldable' $\Rightarrow$ mold (V) + able (A) = moldable (A)
 - -er like in 'singer' $\Rightarrow$ sing (V) + er (N) = singer (N)

Morphology

Inflectional Morphology vs Derivational Morphology

INFLECTION

- 3 In English, all inflectional morphemes are suffixes (i.e. they all only attach to the end of words).
- 4 There can only be one inflectional morpheme per word
- 5 They create new form of same word.

DERIVATIONAL

- 3 There can be multiple derivational morphemes per word
- 4 They can be prefixes, affixes, or suffixes
- 5 They create new words.

Morphology

Inflectional Morphology vs Derivational Morphology

INFLECTION

Examples:

- **Plural:** Bikess, Carss, Truckss, Lionss, Monkeyss, Busess, Matchess, Classess
- **Possessive:** Boy's, Girl's, Man's, Mark's, Robert's, Samantha's, Teacher's, Officer's
- **Tense:** cookedd, playedd, markedd, waitedd, watchedd, roastedd, grilledd; sang, drank, drove
- **Comparison:** Fasterr, Slowerr, Quickerr, Tallerr, Higherr, Shorterr, Smallerr, Weakerr, Strongerr, Sharperr, Biggerr
- **Superlative:** Fastestest, Slowestest, Quickestest, Tallestest, Highestest, Shortestest,

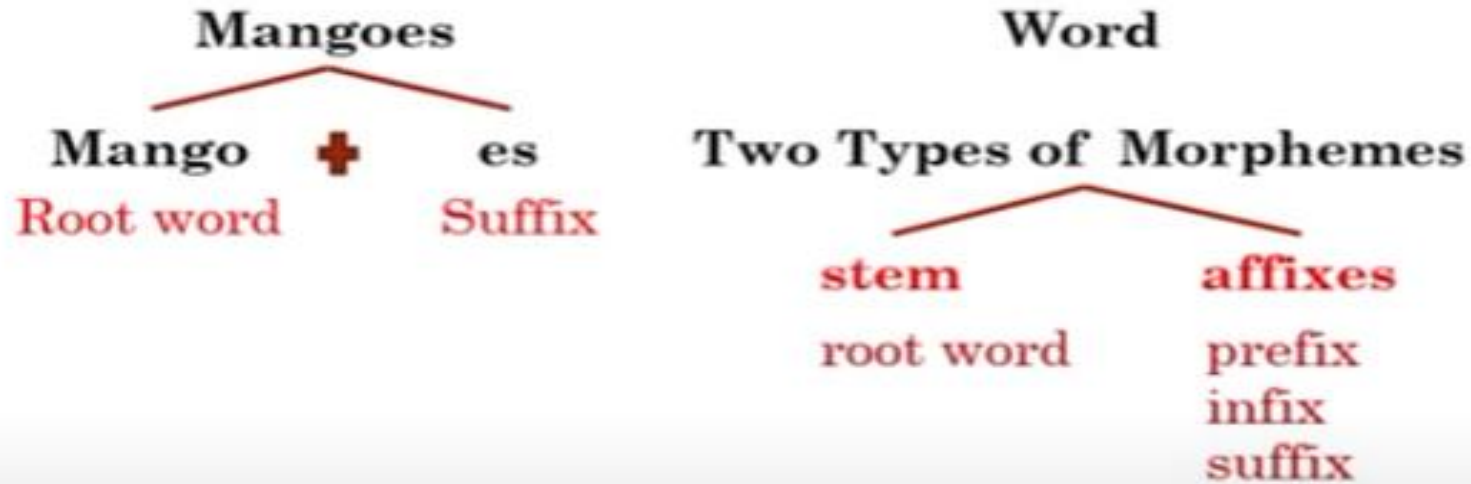
DERIVATIONAL

Examples:

- **Ful:** Beautifulful, Wonderful, Cheerful, Truthful, Tastefful, Flavourful, Joyful
- **Able:** Walkable, Understandable, Loveable, Laughable, Eatable
- **Ment:** Government, Establishment, Agreementment
- **Ness:** Kindness, Truthfulness, Carelessness, Sadness
- **Ly:** Happilyly, Kindlyly, Fortunatelyly, Sadlyly, Livelyly

Morphological Parsing

- Morphological Parsing is used to identify and find the number of morphemes in a given word
- Morphemes are the smallest indivisible meaningful units of a language which builds a word
- For eg.



- Steps to design Morphological Parser
 - Lexicon
 - Morphotactic
 - Orthographic Rules

Morphological Parsing



- Given a word
- Find its root word and Morphological features like:
 - + N (noun)
 - + V (Verb)
 - + A (Adjective)
 - + R (Adverb)
 - + SG (Singular)
 - + PL (Plural)
 - + PRES-PART (Present Participles) - ending with **ing**
 - + PAST-PART

Morphological Parsing

TABLE: Morphological Parsing examples

S.No	Input	Morphological Parse Output
1	cats	cat + N + PL
2	cat	cat + N + SG
3	cities	city + N + PL
4	geese	goose + N + PL
5	goose	(goose + N + SG) or (goose + V)
6	gooses	goose + V + 3SG
7	merging	merge + V + PRES-PART
8	caught	(catch + V + PAST-PART) or (catch + V + PAST)

- First Column – words
- Second column **Stems; Morphological Features**

Morphological Parsing

TABLE: Morphological Parsing examples

S.No	Input	Morphological Parse Output
1	dogs	
2	dog	
3	walked	
4	running	
5	beautiful	
6	quickly	

Morphological Parsing

TABLE: Morphological Parsing examples

S.No	Input	Morphological Parse Output
1	dogs	dog + N + PL
2	dog	dog + N + SG
3	walked	walk + V + PAST-PART
4	running	run + V + PRES-PART
5	beautiful	beauty + A (Adjective) + ful
6	quickly	quick + R (Adverb) + ly

Morphological Parsing

TABLE: Morphological Parsing examples

S.No	Input	Morphological Parse Output
1	ate	
2	eaten	
3	eating	
4	eats	
5	eat	
6	looks	

Morphological Parsing

TABLE: Morphological Parsing examples

S.No	Input	Morphological Parse Output
1	ate	eat + V + PAST
2	eaten	eat + V + PAST-PART
3	eating	eat + V + PRES-PART
4	eats	eat + V + 3SG
5	eat	eat + V + PL
6	looks	look +V +3SG or look +N +PL

Morphological Parsing

We need following to build a Morphological Parser:

- 1 **Lexicon** -List of **Stems** and **Affixes** + Basic Information about them (Noun Stem or Verb Stem)
- 2 **Morphotactics** – **Morpheme ordering Model** (which classes of morphemes follow which other classes)
 - Plural morpheme follows a noun rather than preceding it. e.g.: cats
 $\Rightarrow$ cat + s, dogs
 - decides whether a stem/affix appear or not appear, position of affix (before/after).
- 3 **Orthographic or Spelling Rules** –Spelling changing rules that occur in a word when two morphemes combine
 - $y \Rightarrow$ i.e. (City + s $\Rightarrow$ Cit**ies**) **remove y** and **add ie**.

Morphological Parsing

Lexicon

Stores basic information about a word

Word is stem or a affix?

If stem, then whether a Verb Stem or Noun Stem

If affix, then whether a prefix, infix or suffix

Morphotactic

Set of Rules to make decisions

Decides a word appear/not appear before, after or in between other words

For eg.

use able ness
 ↓
use**able**ness Valid
 obey
 rules

able use ness
 ↓
ableuse**ness** invalid
 disobey
 rules

Orthographic rules

Set of rules used to decide spelling changes

For eg.

lady + s → **ladys**
lady + s → **ladies**
knife + s → **knives**
 correct

Morphological Analysis

Morphology means study of word/making of word

Some words has their own meaning for eg.

Camera Board Pen

Some words are there which when divided into different words, those new words have their own meaning for eg.

Showcase
↙ ↘
Show case

Cocktail
↙ ↘
Cock tail

Some words are there which does not have their own meaning but when they are combined with other words, they become meaningful for eg.

ing
↙ ↘
playing killing

Thus, the detailed study of making of word is called as Morphological Analysis

Finite State Automata

A Finite Automata is defined as $M = \{Q, \Sigma, \delta, q_0, F\}$ where,

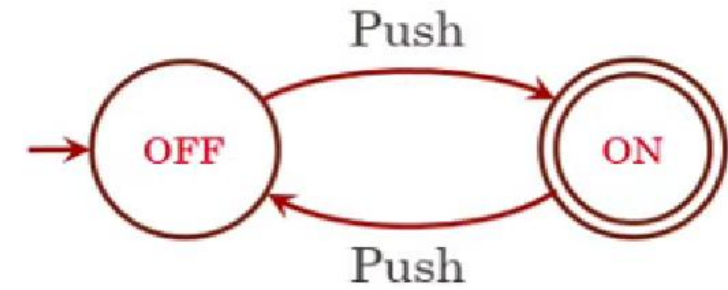
Q : Finite Non-Empty Set of States

Σ : Finite Set of Input Symbols

δ : Transition Mapping: $Q \times \Sigma \rightarrow Q \mid Q \times \Sigma \rightarrow 2^Q$

q_0 : initial State of the Finite Automata

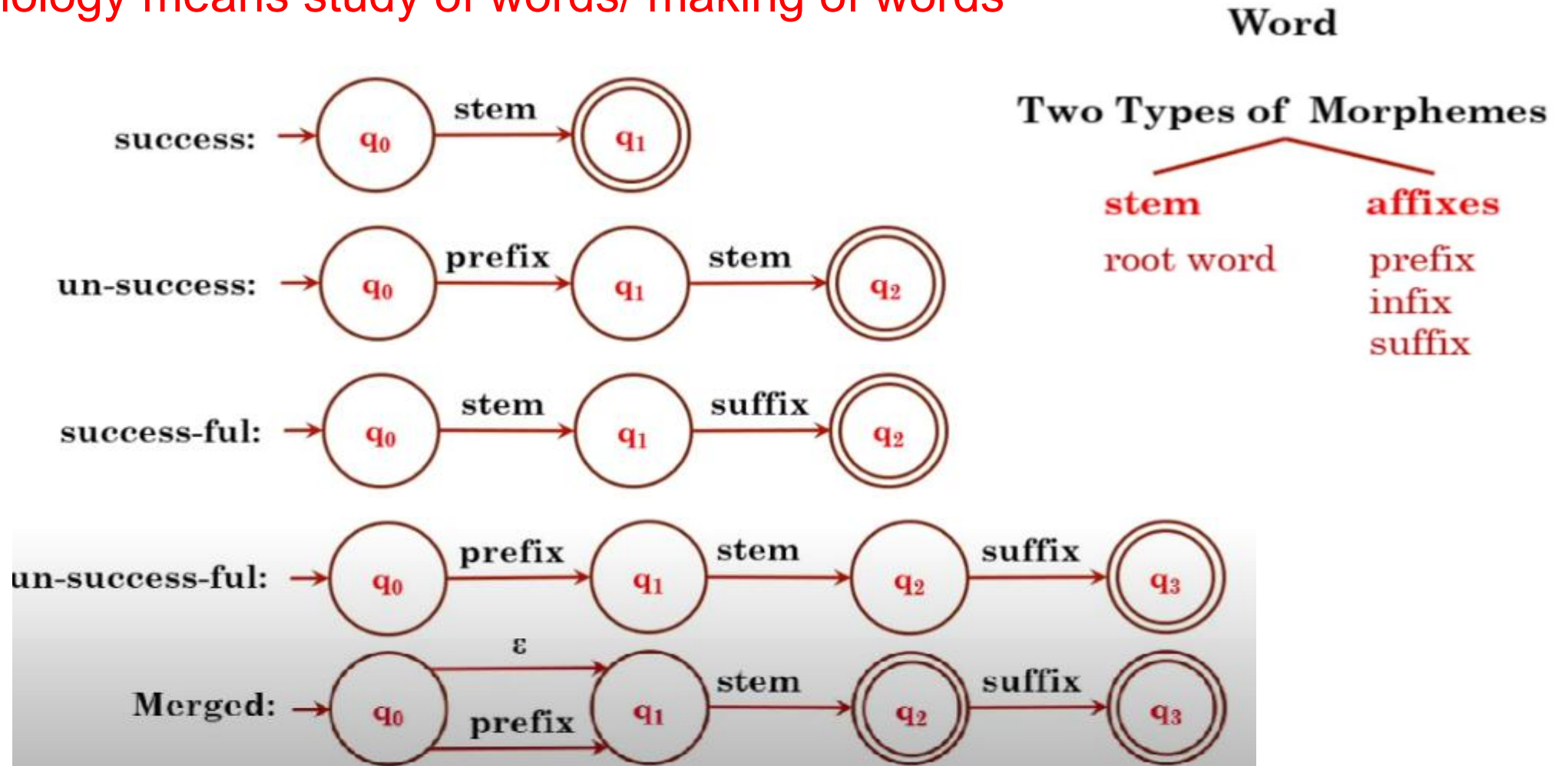
F : Finite Set of Final States



Transition Diagram

Finite State Automata For Morphology

Morphology means study of words/ making of words



How to Model Morphotactics?

One of the simplest way to *model morphotactics* is using **Finite State Automaton (FSA)**.

- **Regular nouns**: that take **regular-s plural**. (cat, dog, kid).
Assumption: Plural form add only **s** as **suffix**. Exception **fox**.
- **Irregular singular nouns**: (goose, mouse)
- **Irregular plural nouns**: (geese, mice)

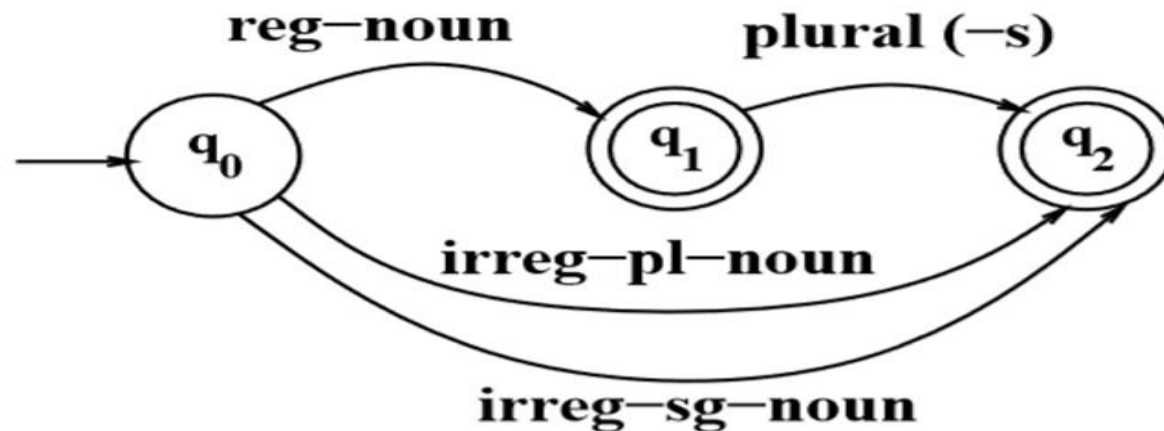


Fig. A Finite State Automata (FSA) for English nominal inflection.

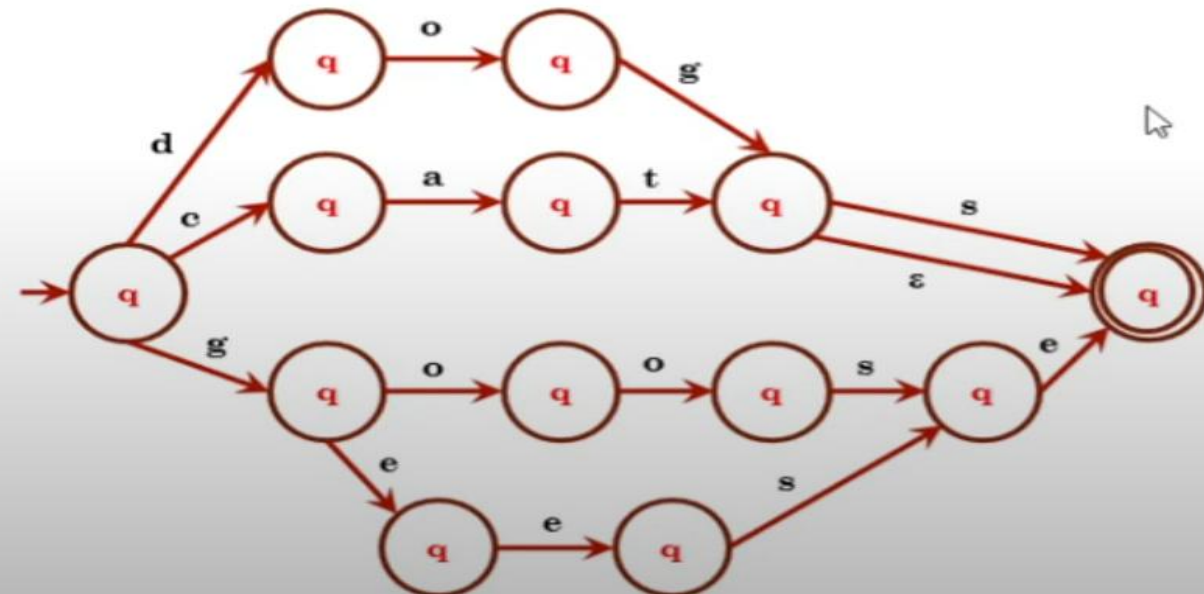
FSA - Verbal Inflection

Stem Changes

- Some irregular word requires stem changes
- For eg.

- goose → geese
- mouse → mice
- teach → taught
- go → went

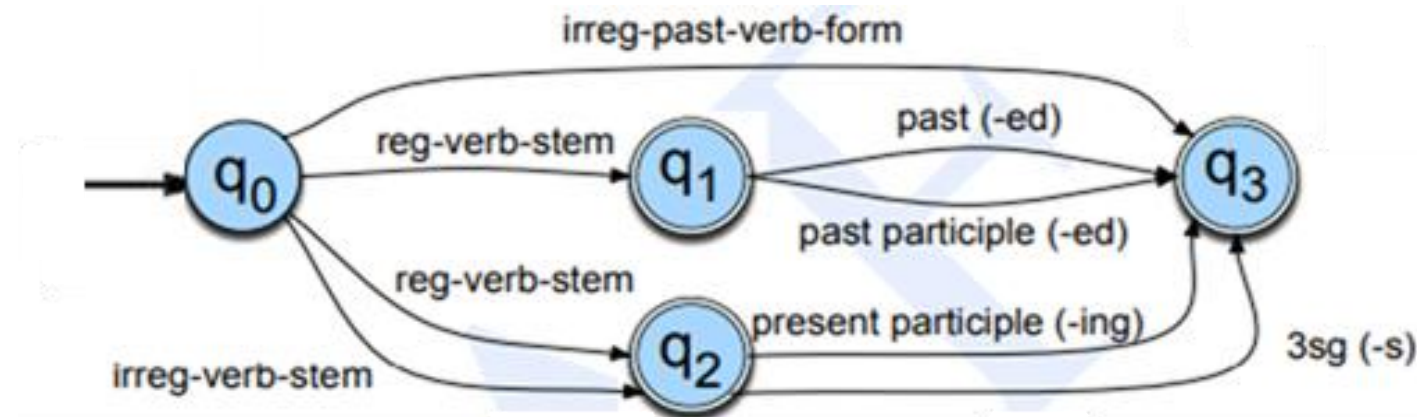
reg.noun	irreg.pl.noun	irreg.sg.noun	plural
fox	geese	goose	-s
cat	sheep	sheep	
dog	mice	mouse	



FSA - Verbal Inflection

- A similar model for English **verbal inflection**
- This lexicon has three stem classes (reg-verb-stem, irreg-verb-stem, and irreg-past verb-form), plus four more affix classes (-ed past, -ed participle, -ing participle, and third singular-s)

reg-verb-stem	irreg-verb-stem	irreg-past-verb	past	past-part	pres-part	3sg
walk fry talk impeach	cut speak sing	caught ate eaten sang	-ed	-ed	-ing	-s

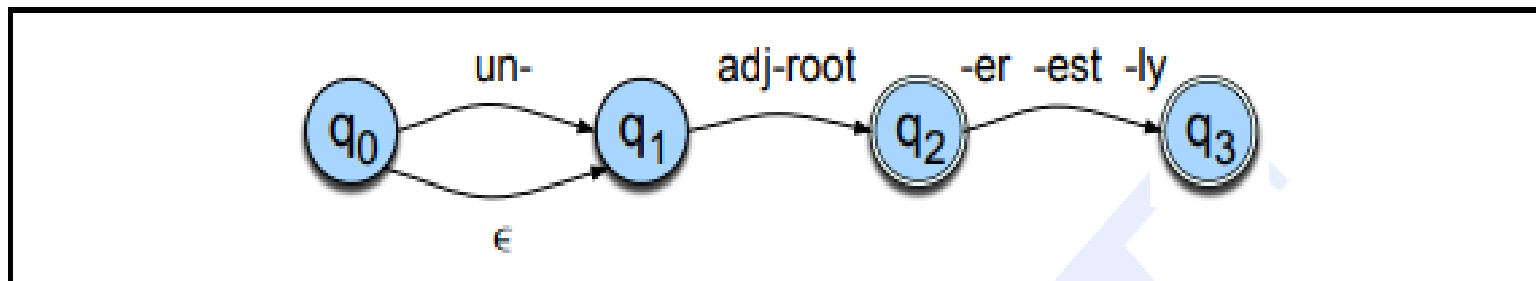


FSA – Adjective Morphology

- **FSA** for a fragment of English **adjective morphology**
- An initial hypothesis might be that adjectives can have an optional prefix (un-), an obligatory root (big, cool, etc.) and an optional suffix (-er, -est, or -ly).
- Here are some examples from Antworth (1990):

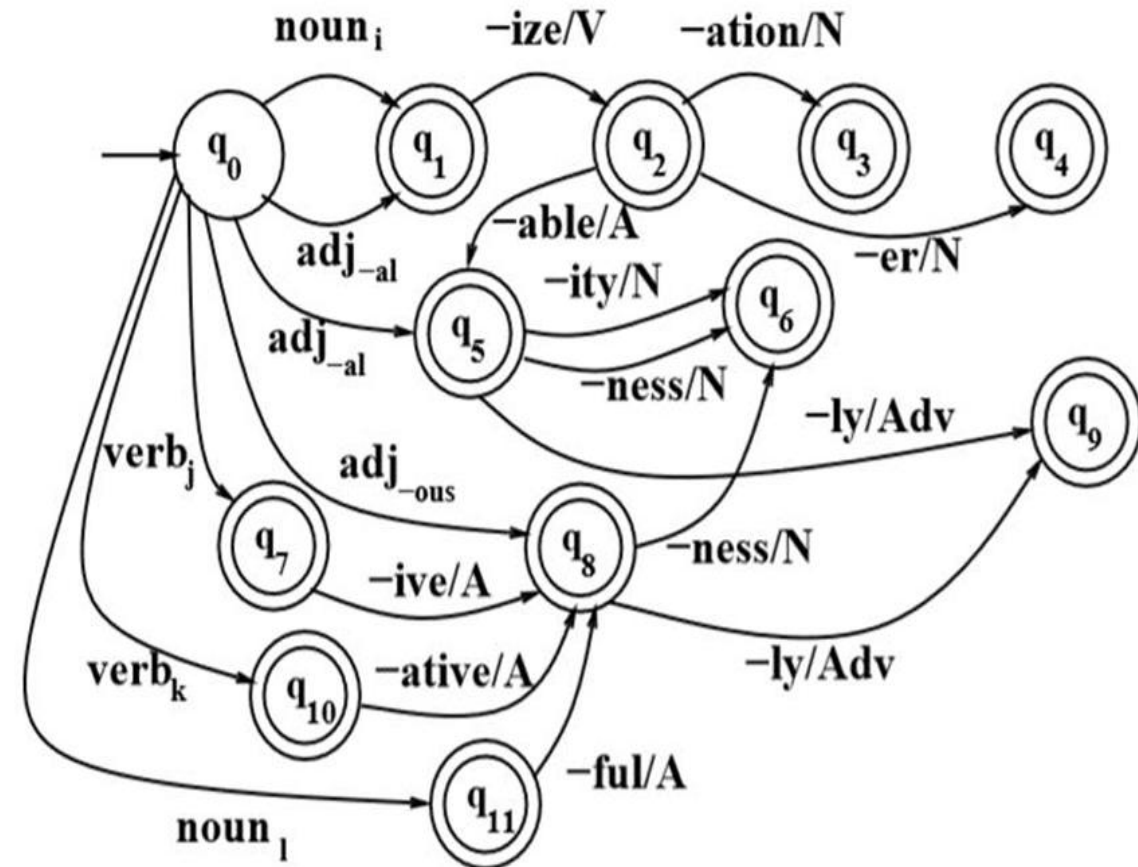
big, bigger, biggest,	cool, cooler, coolest, coolly
happy, happier, happiest, happily	red, redder, reddest
unhappy, unhappier, unhappiest, unhappily	real, unreal, really
clear, clearer, clearest, clearly, unclear, unclearly	

- This might suggest the FSA as shown below:



FSA – Derivational Morphology

- **FSA** for another fragment of English **derivational morphology**.
- Thus since there is a word fossilize, we can predict the word **fossilization** by following **states q0, q1, and q2**.
- Adjectives ending in **-al** or **-able** at **q5** (**equal, formal, realizable**) can take the suffix **-ity**, or sometimes the suffix **-ness** to state **q6** (**naturalness, casualness**).



FSA for Morphological Recognition

- **FSA** can be used to solve the problem of **morphological recognition**.
- Checking if a string of letters is a legal word in English or not
- Procedure:
 - By taking Morphotactic FSAs and plugging in each 'sub-lexicon' into the FSA.
 - i.e. by expanding each arc (eg. **reg-stem-noun**) with all morphemes that make up set of reg-stem-noun.
 - The resulting FSA can be at the level of individual letter.

FSA for Morphological Recognition

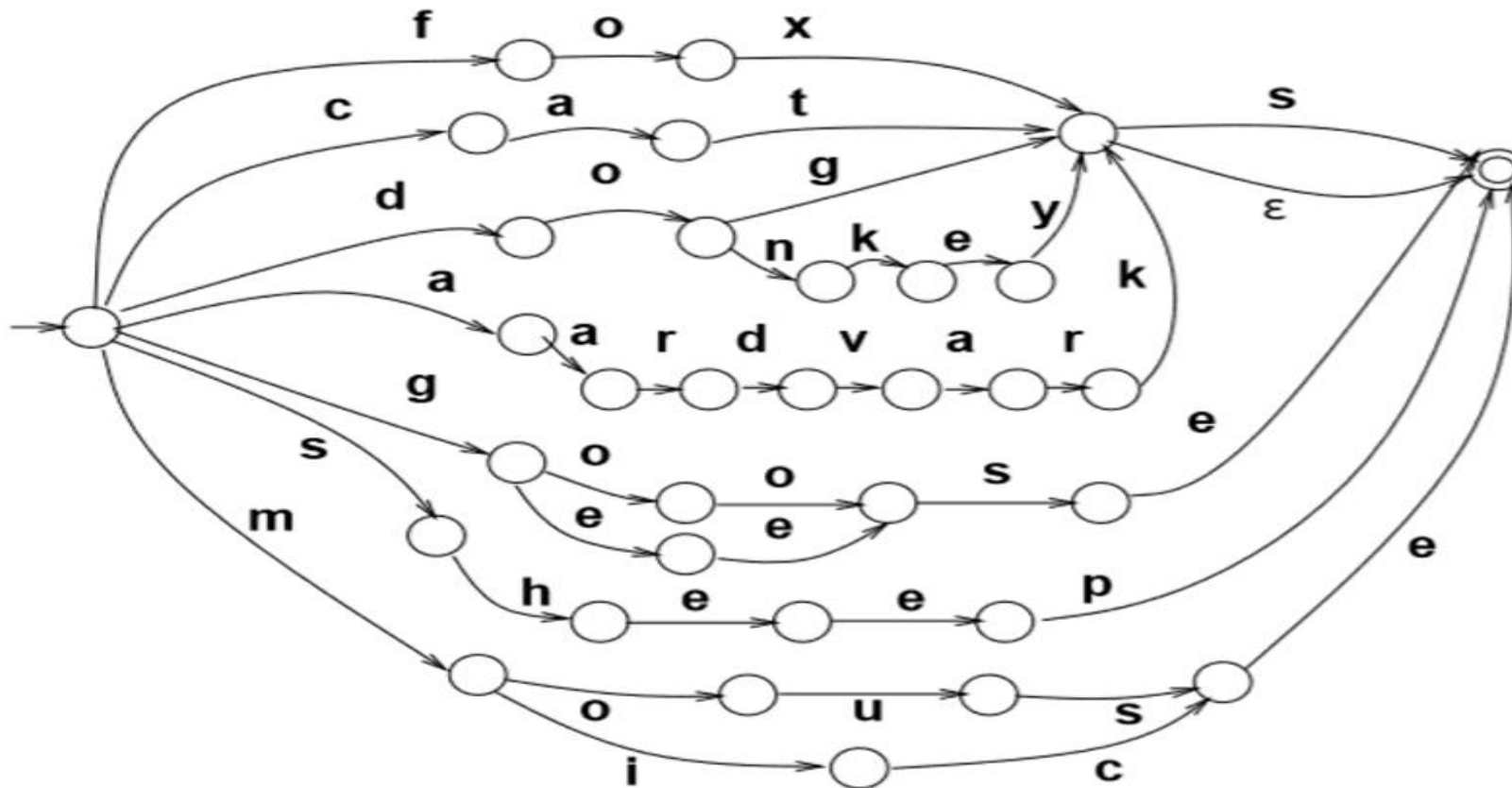


Fig. Expanded FSA for a few English nouns with their inflection.
Note that this automaton will incorrectly accept the input *foxs*.

Finite State Transducers

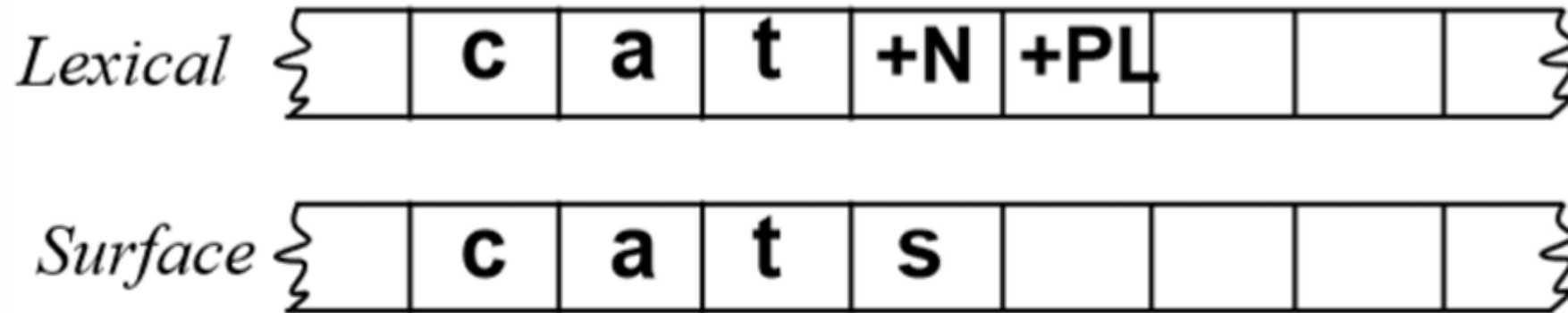
- Two-level Morphology:
 - A word is represented as a correspondence between :
 - **LEXICAL LEVEL** (Simple **concatenation** of morphemes making up a word)
 - **SURFACE LEVEL** (Actual **spelling** of the final word)

Example:

- Cat +N +PL => Lexical Level
- Cats => Surface Level

Finite State Transducers

- Done by building mapping rules between:
 - Feature sequences (lexical level)
 - Letter Sequences (surface level)



Example of the lexical and surface tapes.

FSA used to provide mapping - FST

- A transducer maps between
 - One set of symbols and another
 - FST does this via FSA
- A FST is a two-tape automaton that recognizes (or generates) pairs of strings
- So, **FSA** defines formal languages, by **defining** a set of strings
- **FST** defines **relations** between sets of strings

FST – Mealy Machine Extension to FSA

An FST can be formally defined with 7 parameters:

Q	a finite set of N states $q_0, q_1, \dots, q_{N-1}$
Σ	a finite set corresponding to the input alphabet
Δ	a finite set corresponding to the output alphabet
$q_0 \in Q$	the start state
$F \subseteq Q$	the set of final states
$\delta(q, w)$	the transition function or transition matrix between states; Given a state $q \in Q$ and a string $w \in \Sigma^*$, $\delta(q, w)$ returns a set of new states $Q' \subseteq Q$. δ is thus a function from $Q \times \Sigma^*$ to 2^Q (because there are 2^Q possible subsets of Q). δ returns a set of states rather than a single state because a given input may be ambiguous in which state it maps to.
$\sigma(q, w)$	the output function giving the set of possible output strings for each state and input. Given a state $q \in Q$ and a string $w \in \Sigma^*$, $\sigma(q, w)$ gives a set of output strings, each a string $o \in \Delta^*$. σ is thus a function from $Q \times \Sigma^*$ to 2^{Δ^*}

FSA and FST of Sheep Language

- FSA accepts a language stated over a finite alphabet of single symbols, such as the alphabet of our sheep language:

$$\Sigma = \{b, a, !\}$$

- FST defined this way accepts a language stated over pairs of symbols, as in:

$$\Sigma = \{a : a, b : b, ! : !, a : !, a : \varepsilon, \varepsilon : !\}$$

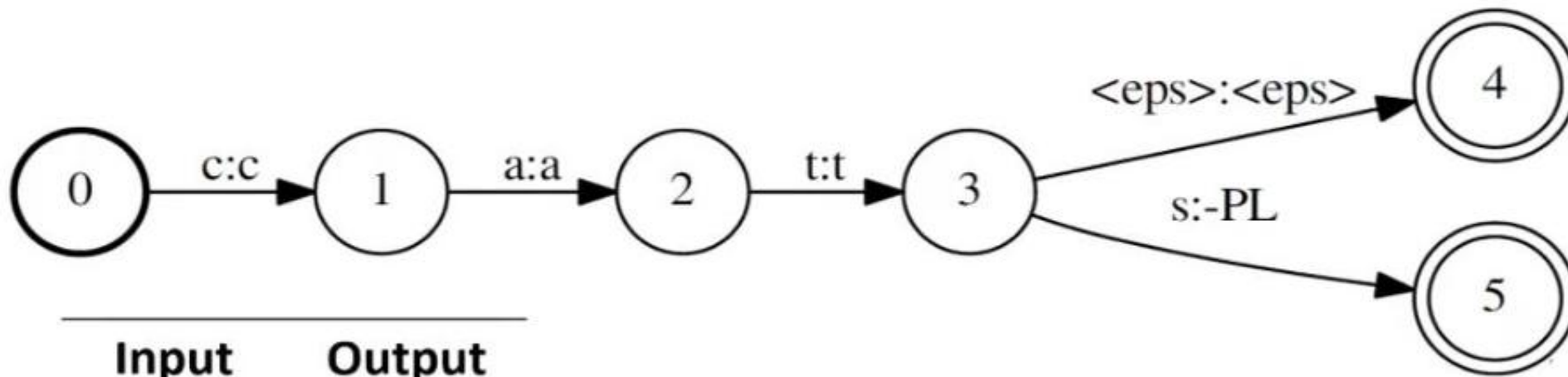
Two-level Morphology (FST with two tapes)

- $a : b \Rightarrow$ Expresses how the symbol a from one tape is mapped to the symbol b on another tape
- $a : \epsilon \Rightarrow$ a on the upper tape will correspond to nothing on the lower tape
- $a : a \Rightarrow$ if a symbol maps to itself, which will be common, it is ***DEFAULT PAIR***
- Upper or **Lexical** Tape
Composed from characters from **LEFT** side of $a:b$ pairs
- Lower or **Surface** Tape
Composed from characters from **RIGHT** side of $a:b$ pairs

Finite State Automata For Morphology

Finite State Transducers

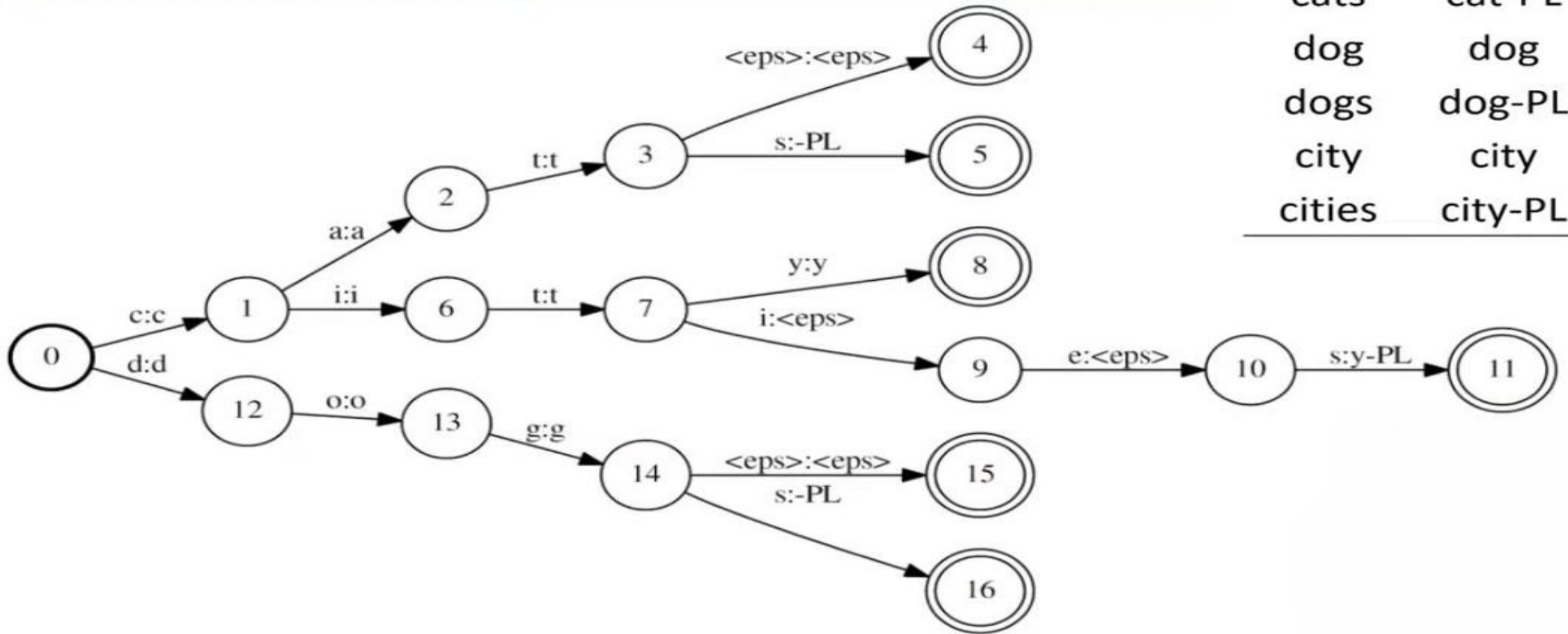
Transducers (FSTs) are similar to finite state machines. They have states, inputs (symbols), and transition between states. They also have **outputs**.



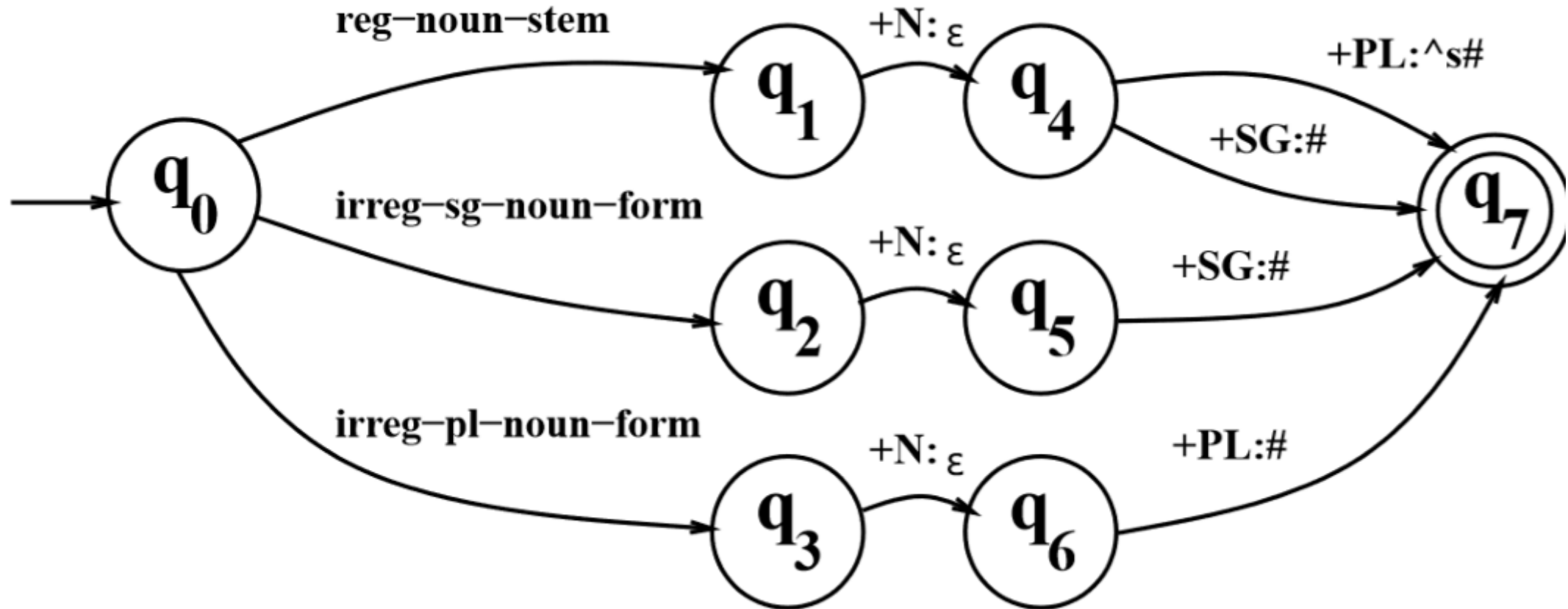
Input	Output
cat	cat
cats	cat-PL

Finite State Automata For Morphology

Finite State Transducers



FST Morphological Parser = Morphotactic FSA + Lexical Tape



A schematic transducer for English nominal number inflection

Lexicon should also have two levels:

Thus the lexicon will look only slightly more complex:

reg-noun	irreg-pl-noun	irreg-sg-noun
fox cat dog aardvark	g o:e o:e s e sheep m o:i u:ɛ s:c e	goose sheep mouse

Cascading Two Automata

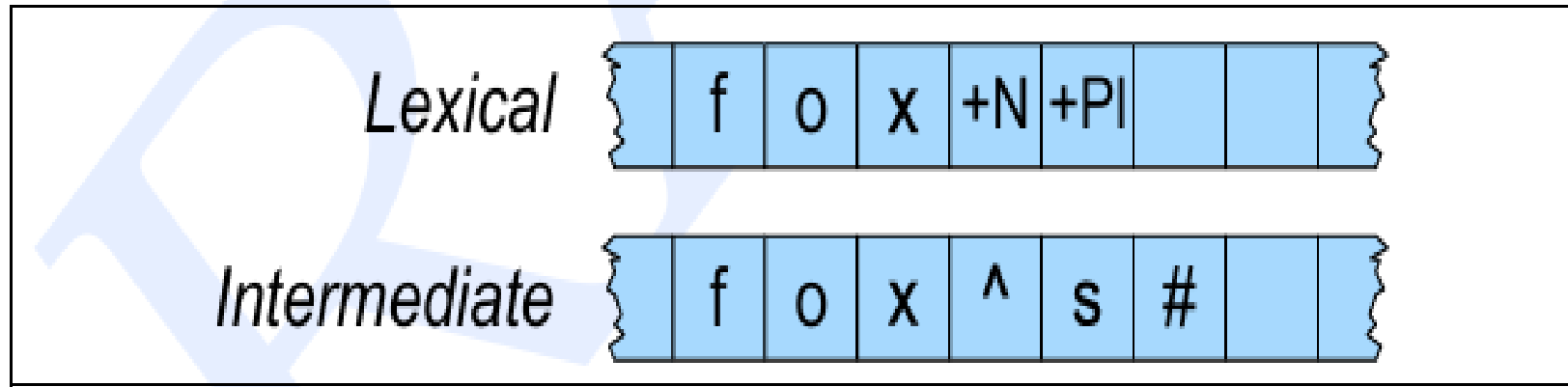
- ▶ *geese => goose +N +PL*
- ▶ Need to Cascade the lexicon (in previous slide) with Singular/Plural Automaton (in slide 38)
- ▶ Cascading two automata means running them in *series* with the *output of first* feeding the *input of second*.

cats => cats +N +PL

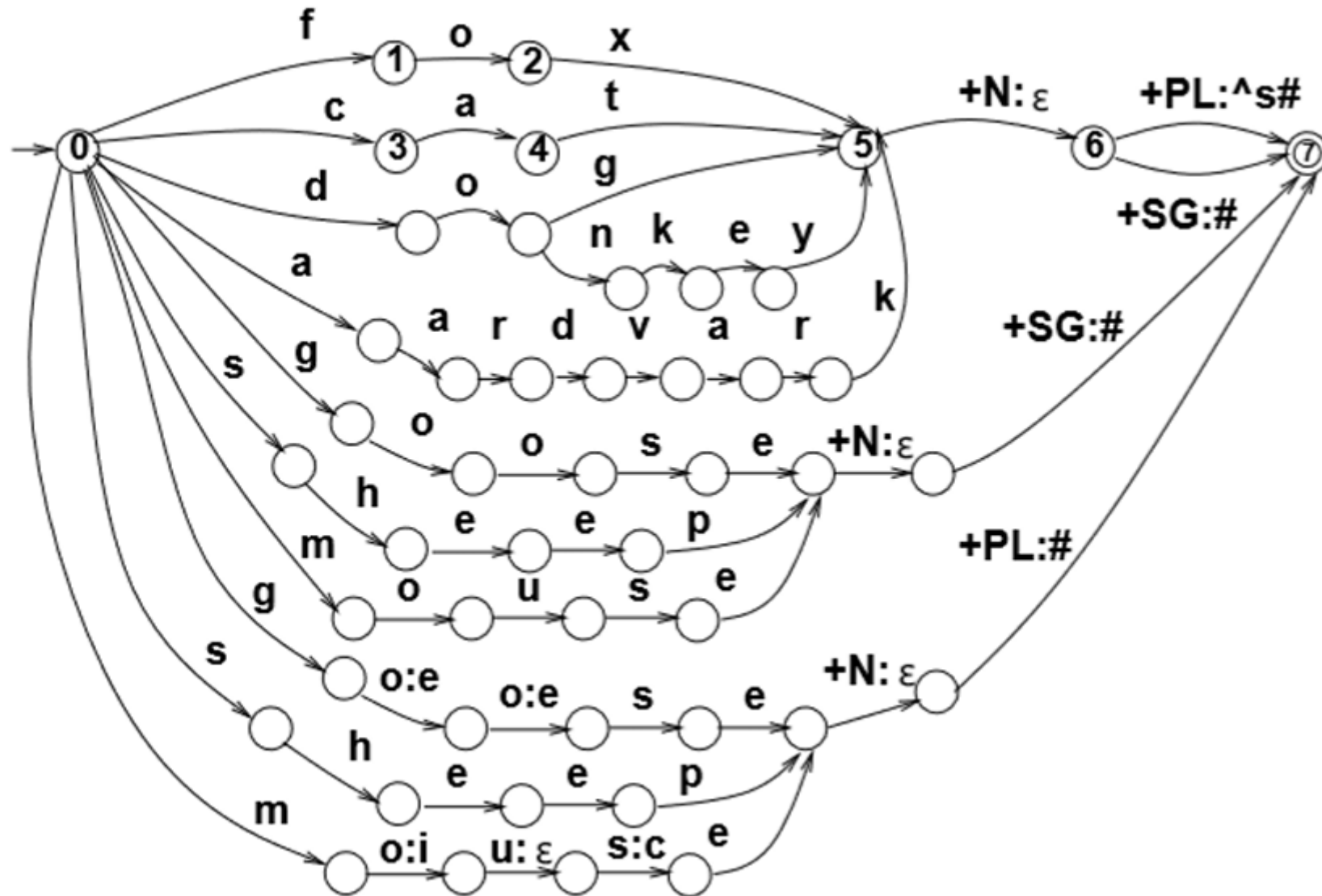
c:c a:a t:t +N: ϵ +PL:^s#

- ▶ *C* maps to **itself** (default pair)
- ▶ Likewise for '*a*' and '*t*'
- ▶ +*N* maps with ϵ transition (Empty String) (+*N* means Noun)
- ▶ +*PL* maps to ^s (^ indicates **Morpheme** boundary)
- ▶ # indicates **Word Boundary**

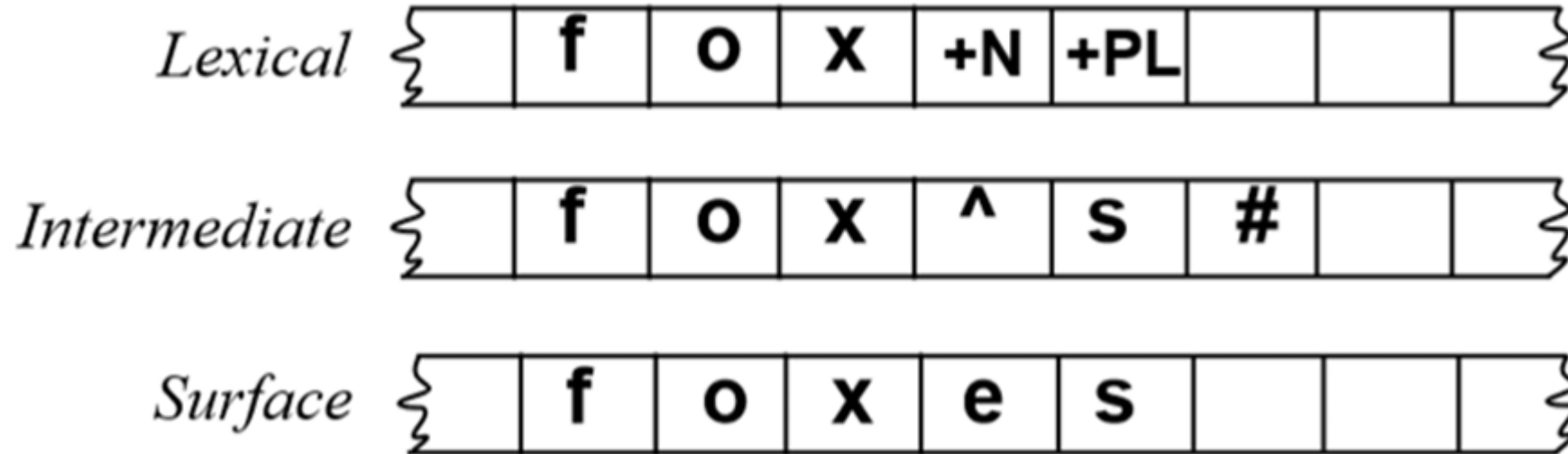
A schematic view of the lexical and intermediate tapes



A fleshed-out English nominal inflection FST



E- Insertion Rule



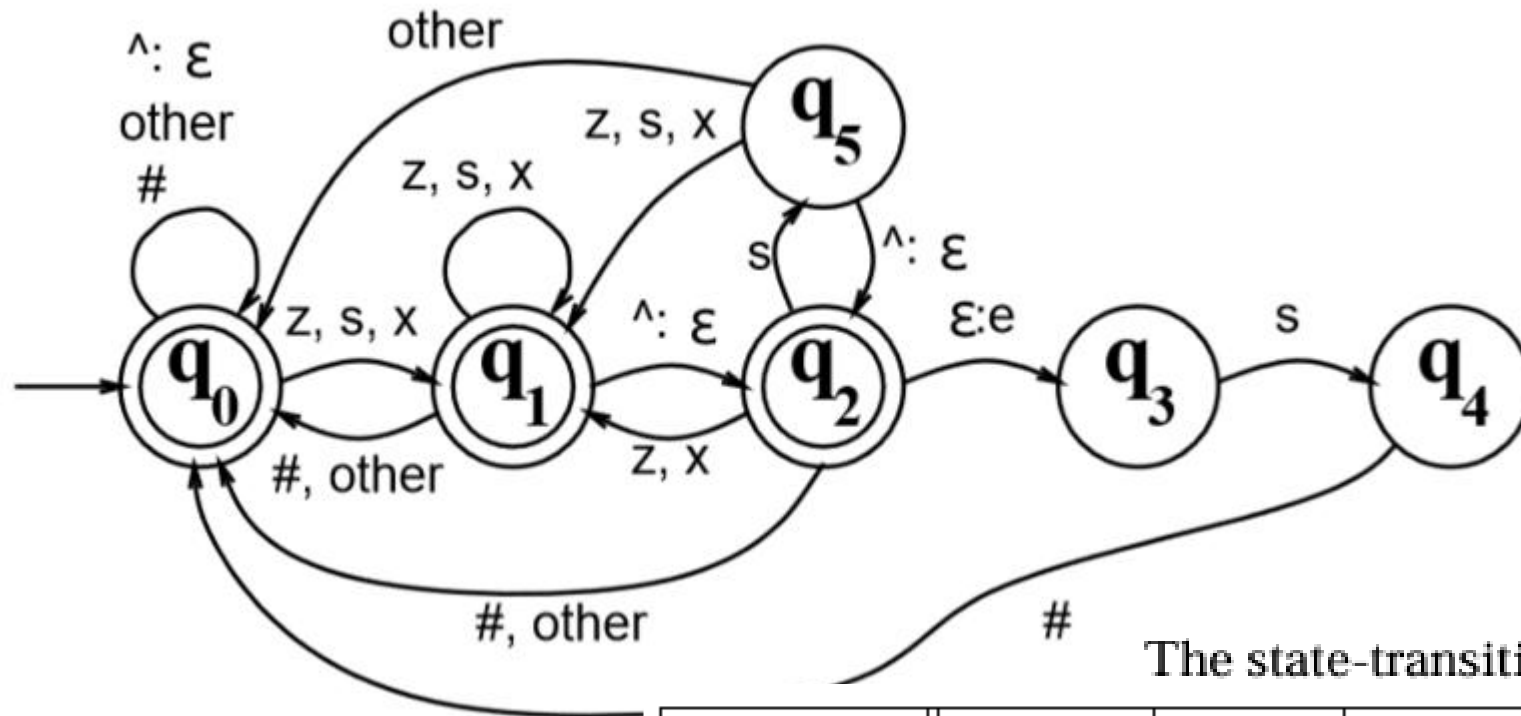
E- Insertion Rule

something like “insert an e on the surface tape just when the lexical tape has a morpheme ending in x (or z , etc) and the next morpheme is $-s$. Here’s a formalization of the rule:

$$\varepsilon \rightarrow e / \left\{ \begin{array}{c} x \\ s \\ z \end{array} \right\} \wedge \text{---} s\#$$

This is the rule notation of Chomsky and Halle (1968); a rule of the form $a \rightarrow b / c \text{---} d$ means ‘rewrite a as b when it occurs between c and d ’

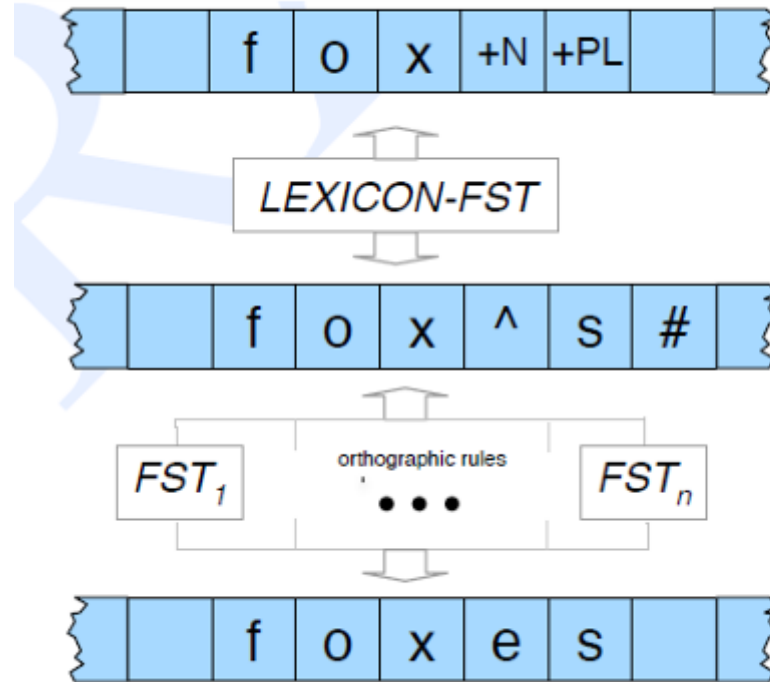
The transducer for the E-insertion rule



The state-transition table for E-insertion rule

State \ Input	s : s	x : x	z : z	$\wedge : \epsilon$	$\epsilon : e$	#	other
q_0 :	1	1	1	0	-	0	0
q_1 :	1	1	1	2	-	0	0
q_2 :	5	1	1	0	3	0	0
q_3 :	4	-	-	-	-	-	-
q_4 :	-	-	-	-	-	0	-
q_5 :	1	1	1	2	-	-	0

Combining FST Lexicon And Rules



Combining FST Lexicon And Rules

